Finding God's Number for the Rubik's Cube Using Computational Analysis

By

Ryan Taylor McGregor

Submitted to the Department of Computer Science
In Partial Fulfillment
Of the Requirements for the Degree of

Masters of Science

in

Computer Science

at

Hofstra University

2016

Professor Thomas Re, Advisor

Dr. Gerda Kamberova, Thesis Committee Member

Dr. Krishnan Pilliapakkmnatt, Thesis Committee Member

Dr. Chuck C. Liang, Graduate Coordinator, Dept. of Computer Science

ProQuest Number: 10253657

All rights reserved

INFORMATION TO ALL USERS
The quality of this reproduction is dependent upon the quality of the copy submitted.

In the unlikely event that the author did not send a complete manuscript
and there are missing pages, these will be noted. Also, if material had to be removed,
a note will indicate the deletion.

ProQuest 10253657

Published by ProQuest LLC (2017). Copyright of the Dissertation is held by the Author.

All rights reserved.
This work is protected against unauthorized copying under Title 17, United States Code
Microform Edition © ProQuest LLC.

ProQuest LLC.
789 East Eisenhower Parkway
P.O. Box 1346
Ann Arbor, MI 48106 – 1346

Abstract

The purpose of this thesis is to explore possible solution methods to the God's Number problem for the 3x3x3 Rubik's Cube using the Quarter Turn Slice Metric. Topics in this thesis subject include the history of the Rubik's Cube and the God's Number problem, various metrics, an analysis of how many moves it takes to solve cube sequences of various distances, and a more mathematical approach to solving God's Number based on my initial approaches' results. In more detail, there is a breakdown of the process that led me to build the logic for my initial approach, and how the derived results led me to a future approach.

From reading this thesis, you will gain a firm understanding of the thought process that went behind this approach towards solving God's Number, as well as the differences between this approach and that chosen by others in the past.

Dedication

I would like to dedicate this thesis to the following people; my mother, Lori McGregor, my father, Ken McGregor, my brother, Matt McGregor, my sister, Nicole DeRuvo, my colleague and friend, Ben Avchen, and my girlfriend, Melanie Pavlidis. Without the patience and support of everyone named, this thesis wouldn't have been possible.

Acknowledgements

I would like to acknowledge my thesis advisor, Professor Thomas Re for the patience, hard work, and dedication in helping me with this undertaking. The expertise, guidance, and friendship provided throughout this process has been invaluable, and without it this thesis would not have been feasible.

I would also like to acknowledge my committee members, Dr. Krishnan Pilliapakkmnatt, Dr. Gerda Kamberova, the department chair, Dr. Chuck C. Liang, and all the faculty and staff in the Computer Science department at Hofstra University. Without the dedication and support of all those I've mentioned, I would not have had the knowledge, insight, or expertise to make this thesis a reality.

Table of Contents

Chapter 1 – Introduction to the God's Number Problem

- **1.1** - History of the Rubik's Cube
- **1.2** - History of the God's Number Problem
- **1.3** - How It Was Solved
- **1.4** - Problems with Solution Method (From a Mathematical Standpoint)
- **1.5** - A New Approach

Chapter 2 – Computational Approach to Solving for God's Number

- **2.1** - My Initial Approach
  - **2.1.1** – Metrics
  - **2.1.2** – Nomenclature
- **2.2** – Apply My Approach to the 2x2x2 as a Proof of Concept
- **2.3** – Problems with My Initial Approach

Chapter 3 – Results to My Initial Approach

- **3.1** – Results of My 2x2x2 Program
- **3.2** – Findings of My Initial Approach

Chapter 4 – Next Approach

- **4.1** – Expansion into Mathematical Analysis
  - **4.1.1** – Repeats
- **4.2** – How to Next Approach the God's Number Problem
- **4.3** – Cycles

Chapter 5 – Conclusion

- **5.1** – Results
- **5.2** – Potential Applications of Findings

Chapter 1 – Introduction to the God's Number Problem

1.1 - History of the Rubik's Cube

The Rubik's Cube was invented by Erno Rubik, a Hungarian Professor of architecture, in 1974. He designed and built the cube in an attempt to try and show his students a better way of understanding 3D geometry and spatial relationships [8]. After first creating the cube and mixing it up, he wasn't able to solve it for more than a month, finally succeeding by using a method of rearranging the corners of each side first [7].

Being behind the infamous 'Iron Curtain' that existed during the Cold War, the cube wasn't able to go mainstream and become an international toy sensation until it was licensed to the Ideal Toy Corp in 1980 after being shown at the Nuremberg Toy Fair in 1979. It went on to win Toy of the Year in the UK in both 1980 and 1981, and has since gone on to sell an estimated 350 million units worldwide [7]. It has inspired many things to come out of pop culture since 1980, including movies, TV shows, art, games, and even a new sport, 'speed cubing'.

1.2 – History of the God's Number Problem

As 'speed cubing' became more mainstream, hobbyists, competitors, and mathematicians alike began investigating the Rubik's Cube to see if they could find ever more efficient ways of solving it. For hobbyists and competitors, the search was a means to a faster way of solving the cube, either to impress onlookers, or to perform better during competition. Mathematicians, on the other hand, shared Ernos' love of spatial relationships and understood that despite its seeming simplicity and disposition as a toy, there are some rather serious mathematics involved in fully understanding all of the Rubik's Cubes' complexities.

The Rubik's Cube has been found to have 43,252,003,274,489,856,000 possible states it can exist in. This can also be written as:

$$8! * 3^7 * \left(\frac{12!}{2}\right) * 2^{11} = 43{,}252{,}003{,}274{,}489{,}856{,}000$$

Figure 1.2.1 – Equation for total states possible on a 3x3x3 Rubik's Cube.

In that equation, $8!$ represents all the ways to arrange the 8 corner cubes. 3^7 represents all the possible orientations of those 8 corners, as each corner has 3 sides. It turns out that only 7 of the corners can be oriented independently, and the orientation of the eighth depends on the orientation of the preceding seven. There are 12 edges on a 3x3x3 Rubik's Cube, so one would think there are $12!$ different ways to arrange the edge cubes, but the reality is that it's $\frac{12!}{2}$. This has to do with the parity of a permutation. "The parity of a permutation refers to whether that permutation is even or odd. An even permutation is one that can be represented by an even

number of swaps while an odd permutation is one that can be represented by an odd number of swaps [16].” To clarify this idea, one must understand that “every cube state reachable by legal moves can always be represented by an even number of swaps, and at the same time cannot be represented by an odd number of swaps (the two are mutually exclusive). To understand why this is so, we need to realize that each legal move always performs the equivalent of an even number of swaps. No matter how many moves you perform, the number of accumulated swaps will therefore always remain even. For example, consider the turning of one face by 90 degrees:

```
C1  E1  C2       C4  E4  C1
E4      E2  -->  E3      E1
C4  E3  C3       C3  E2  C2
```

Figure 1.2.2 – A sample 90° face turn to demonstrate parity.

The new corner state can be obtained via 3 swaps (swap C1/C4, swap C1/C3, swap C1/C2). Similarly, the new edge state can be obtained via 3 swaps. All together, this is 6 swaps which is even. Therefore, no matter how many moves you perform, always an even number of swaps will have been performed [15]”. Essentially, because odd parities are impossible without physically taking the cube apart and reassembling it with a different parity, we can eliminate half of the 12! states the edges can be in, since edges and corners must have the same parity, which means that the entire cubes parity will always be even. Finally, 2^{11} represents the orientation of the 12 edge pieces. Like the corners, the last edge’s orientation is dependent on the orientation of the preceding 11 [12].

Figuring out how to optimally solve the Rubik’s Cube from a random position became an important goal for most everyone that took more than a passing interest in the cube, but being able to do so for all 43 quintillion+ cases amounted to a Herculean task. Instead, simply finding the smallest number of moves it would take in the ‘worst case’ scenario, that is, when the cube is the furthest away it can possibly be from being solved, became an endeavor in its own right. This number, the most amount of moves it should ever take to solve a Rubik’s Cube, presuming you’re solving it optimally, was dubbed God’s Number, which is a nod to the notion that it would take a God-like entity to be able to do this consistently given the cube’s huge amount of possible states.

To find God’s Number, the problem was approached by forming lower and upper bounds for what God’s Number could be, and finding ways to whittle away at those bounds until eventually the number could be known. In 1980, a lower bound of 18 had been established by analyzing the number of effectively distinct move sequences of 17 or fewer moves, and finding that there were fewer such sequences than Cube positions. The lower bound wasn’t increased until 1995, when Michael Reid proved that the “superflip” position (corners correct, edges placed correctly but oriented incorrectly) required 20 moves.

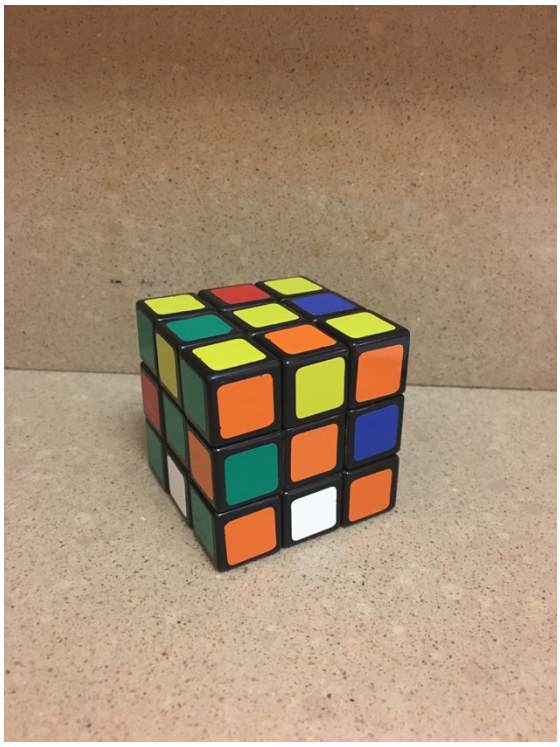


Figure 1.2.3 – The "Superflip" position.

The upper bound was shown to be at least 52 in 1980 by Morwen Thistlethwaite, who developed a complicated algorithm that was always able to solve the cube in 52 moves or less. By 1995 the upper bound had been lowered to 29 due to improvements in Thistlethwaite's algorithm. After that progress on the problem slowed until about 2005, when rapid developments in computational power and taking advantage of mirroring and symmetry allowed researchers to use Kociemba's "two-phase" algorithm to check every state necessary in order to prove that God's Number was 20. It should be noted here that this finding is for what is called the Half Turn Metric, or HTM, which means that turning a face 180° counts as only 1 move. In 2014, using similar strategies, it was shown that God's Number for the Quarter Turn Metric, or QTM, is 26 [1].

1.3 – How It Was Solved

As was mentioned earlier, the general strategy for finding God's Number was to build an algorithm that optimally, or very nearly optimally solved the Rubik's Cube, and then run that algorithm on every possible state to show that it will never take more than X moves to solve.

The first major algorithm was created by a mathematician named Morwen Thistlewaite. His algorithm was initially able to always solve the cube in a maximum of 52 moves. It worked by not solving the cube one piece or one layer at a time, but instead by attacking all the pieces at the same time, and restricting them to fewer and fewer possibilities until there is only one position left for each piece and the cube is solved. It would start by doing a few moves until a

position arises that can be solved without using quarter turns of the up and down faces, (referenced as the U and D faces respectively) though half turns of U and D are still needed. It then proceeds to solve the cube without using U or D quarter turns by first moving to a position that does not need quarter turns of the front and back faces either (F and B). With these further restrictions a position is arrived at that does not need any quarter turns at all, and can hence be solved by half turns only. The cube then indeed gets solved using half turns only [10].

After some improvements to Thistlewaite's algorithm lowered the upper bound to 45, Herbert Kociemba managed to combine several ideas into a very effective new algorithm, based on Thistlewaite's algorithm except more efficient. Unlike Thistlewaite's algorithm, which had 4 phases, or steps, Kociemba's used only 2. Each of Kociemba's phases combines two of Thistlewaite's phases together. The first phase of Kociemba's algorithm finds the shortest possible way of going through the first two phases of Thistlewaite's, and the second phase finds the shortest way to go through the last two phases of Thistlewaite's algorithm. The phases have actually been fully calculated and their maximum lengths were found to be 12 and 18 moves, so the algorithms first solution is at most 30 moves long. It was then proven that the 29 and 30 move cases can be avoided, so that every cube position can be solved in at most 28 moves. Using similar calculations, over the next couple of years, mathematicians were able to get the upper bound as low as 22, by accounting for certain sets within use of Kociemba's algorithm and showing that they never need more than a certain amount of moves to complete a phase.

Finally, in 2010, Tomas Rokicki, Herbert Kociemba, Morley Davidson, and John Dethridge were able to prove that the upper bound was also 20, proving that God's Number for the Cube is exactly 20. They did this by first partitioning the positions of the cube into 2,217,093,120 sets of 19,508,428,800 positions each. Next they reduced the count of sets they needed to solve to 55,882, 296 using symmetry and set covering. Since the lower bound was 20 already, to prove God's Number was in fact 20, they didn't need to find optimal solutions to each position, but instead only solutions of length 20 or less. They then wrote a program that solved a single set in about 20 seconds. At approximately 3 sets a minute, it took about 35 CPU years to find solutions to all of the positions in each of the 55,882,296 sets [1].

1.4 – Problems with Solution Method (From a Mathematical Standpoint)

While the culmination of 30 years of working on this problem has produced an impressive result, from a mathematical point of view, it does leave some questions left unanswered. Given the fact that the researchers needed 35 years of CPU time just to brute force this answer, how long exactly would a similar approach take to find God's Number for a 4x4x4 Rubik's Cube? What about a 5x5x5, or NxNxN? Clearly, for anything larger than a 3x3x3, this approach isn't practical. Also, this approach teaches us precious little about why, exactly, God's Number is 20 for the 3x3x3 Rubik's Cube. By taking a brute force approach, opposed to a mathematical one, there is almost no mathematical understanding to the result that God's Number is 20. As a mathematician and cube enthusiast, having no context to help explain why God's Number is 20, and no path based on this work to attempt to find God's Number for any larger Rubik's Cubes, is extremely frustrating.

1.5 – A New Approach

As I read the cube20.org page that describes the proof of how the researchers managed to figure out that God's Number was 20, I wondered to myself how it can be a formal proof if the algorithm they used wasn't proven to solve a cube optimally. Of course I later found the clarification that it didn't need to be optimal, it only needed to be 20 or less to fulfill their proof, but asking that question led me to a more important one; If I didn't have an algorithm that can solve the cube (optimally or not), how else might I go about attempting to prove that God's Number is 20?

There is a chart on the cube20.org page that, while incomplete, shows the count of positions relative to distance from being solved. After posing myself that question, I saw that chart and couldn't find any concrete pattern. That got me thinking that since the Rubik's Cube is a structure that follows rules, there must be some type of pattern that is just too complex for me to initially understand, and if that structure could be understood, it could be used to complete the chart. Completing that chart with certainty would absolutely be an alternate proof that God's Number is 20, as it would show that when you add all the counts of positions at each distance from 0 to 20, the sum is equal to the total amount of possible cube states, 43,252,003,274,489,856,000. More importantly, finding a pattern that describes how the counts of positions at each distance are formed could, potentially, lead to generalizing that pattern to find God's Number not just for a 3x3x3 Rubik's Cube, but a cube of any size.

Distance	Count of Positions
0	1
1	18
2	243
3	3,240
4	43,239
5	574,908
6	7,618,438
7	100,803,036
8	1,332,343,288
9	17,596,479,795
10	232,248,063,316
11	3,063,288,809,012
12	40,374,425,656,248
13	531,653,418,284,628
14	6,989,320,578,825,358
15	91,365,146,187,124,313
16	about 1,100,000,000,000,000,000
17	about 12,000,000,000,000,000,000
18	about 29,000,000,000,000,000,000
19	about 1,500,000,000,000,000,000
20	about 490,000,000

Figure 1.5.1 – Chart displaying the amount of cube states that exist at each distance from solved.

Chapter 2 – Computational Approach to Solving for God's Number

2.1 – My Initial Approach

2.1.1 – Metrics

Before I continue, at this point I should make clear that for my approach, I chose not to use the popular Half Turn Metric (HTM for short), which is what researchers used when they proved that God's Number is 20 (they've also proven that for the Quarter Turn Metric, QTM for short, God's Number is 26), but to instead use what is called the Quarter Turn Slice Metric (QTSM). For clarification, I'll briefly summarize the different metrics out there:

HTM; Half Turn Metric (or Face Turn Metric): Any face turn of any angle counts as 1 turn.

QTM; Quarter Turn Metric: Only quarter (90°) face turns count as 1 turn.

STM: Slice Turn Metric: Any turn of any layer, by any angle, counts as 1 turn.

QTSM; Quarter Turn Slice Metric: Only quarter turns of any layer, counts as 1 turn.

ATM; Axial Turn Metric: Any movement within the same axis counts as 1 turn.

ETM: Execution Turn Metric: Any perceived movement counts as 1 turn.

BTM; Block Turn Metric: A metric for measuring move-counts on Big cubes [9]

I chose to use the QTSM for a couple of reasons. While the HTM metric is the most popular metric among speed cubers, from a mathematical standpoint it doesn't make the most sense to use. Firstly, the idea of counting a 180° turn as one move, when it is the equivalent to two separate 90° moves, is troubling to me. Secondly, the HTM doesn't count anything other than face turns. To me, when I hear the phrase 'optimal', I want to see a cube solved in the fewest moves possible, so the idea that I'd have to turn two opposite faces in the same direction to emulate turning the slice between them only once in the opposite direction seemed illogical (See Figure 2.1.1.1). Finally, I chose the QTSM because on the whole, my approach, if successful, will provide a solid foundation for attempting to find God's Number for larger Rubik's Cubes', and when there are more than 1 middle layers, using any metrics that only utilize face turns quickly become obsolete.

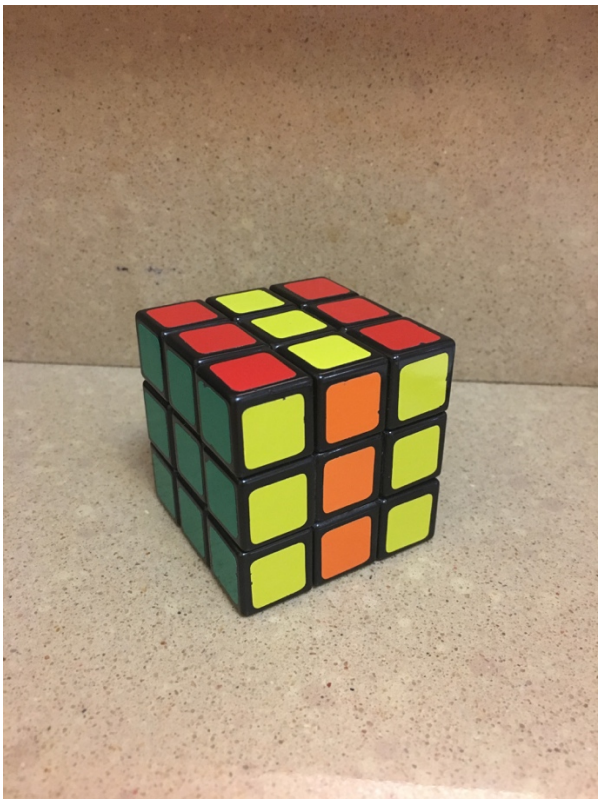


Figure 2.1.1.1 – The given cube state can be achieved by performing either of the following sequences; left and right prime (written as LR'), or M', which is the counterclockwise rotation of the mid-slice between the left and right faces (counterclockwise relative to the left face). M' is a move that isn't allowed in the HTM.

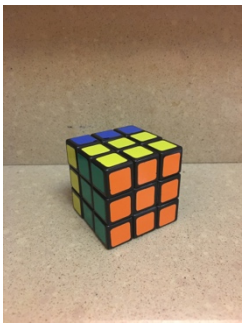
After deciding on my metric, I was delighted to find out that no one has proven God's Number for it yet, so if I end up being successful in finding God's Number with this approach, it will be a new finding, even if it does still end up being 20, which I strongly suspect, given that my approach and the HTM's approach each allow the cube to have 18 different possible moves.

2.1.2 – Nomenclature

Also before moving forward, I need to briefly discuss the available moves for the QTSM, and how I'll be referring to them going forward. There are 18 total possible moves when using the QTSM, the 6 faces can each go clockwise or counterclockwise 90°, and the 3 slices between each pair of faces on opposite sides of the cube can also each go clockwise or counterclockwise 90°. Here are the 18 moves with brief explanations for each:

F – a front face clockwise 90° turn

F' – a front face counter-clockwise 90° turn



B – a back face clockwise 90° turn

B' – a back face counter-clockwise 90° turn

L – a left face clockwise 90° turn

L' – a left face counter-clockwise 90° turn

R – a right face clockwise 90° turn

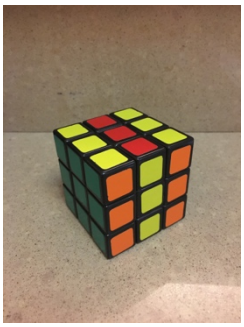
R’ – a right face counter-clockwise 90° turn

U – an up face clockwise 90° turn

U’ – an up face counter-clockwise 90° turn

D – a down face clockwise 90° turn

D’ – a down face counter-clockwise 90° turn



M – a mid-slice (between left and right faces) clockwise (as if it were a left face) 90° turn

M’ - a mid-slice (between left and right faces) counter-clockwise (as if it were a left face) 90° turn

E – a mid-slice (between up and down faces) clockwise (as if it were a down face) 90° turn

E' – a mid-slice (between up and down faces) counter-clockwise (as if it were a down face) 90° turn

S – a mid-slice (between front and back faces) clockwise (as if it were a front face) 90° turn

S' – a mid-slice (between front and back faces) counter-clockwise (as if it were a front face) 90° turn

Figure 2.1.2.1 – Images and descriptions of the 18 possible moves on a 3x3x3 Rubik's Cube using the Quarter Turn Slice Metric (QTSM).

Rather than attempting to solve every necessary state to show that they can all be solved in 20 or less moves, my approach is to instead account for all 43 quintillion+ moves by calculating how many states exist at each distance from solved, from 0 up until the sum of the total number of states at each distance equals the total number of possible cube states.

Since there are 18 moves with the QTSM metric, when first viewing the chart from Figure 1.5.1, it made sense to begin trying to account for the data in it by assuming that the count of positions is simply 18^x, where x is the distance. When x = 0, the count is 1, and when x = 1, the count is 18, so at least in the beginning this holds true. However, 18^2 = 324, not 243, so in order to write a program that can figure out the count of positions, I needed to figure out why there are only 243 positions that have a distance equal to 2, instead of 324, and then build the logic that results in that 243 into my program. I realized that because the chart from Figure 1.5.1 is based on the HTM, its possible that my eventual results will not match it. Despite that, it still made sense to do an analysis of the 2-move sequences in order to figure out the logic of my program, and at least use the chart as a starting point. I decided to go through all 324 possibilities of 2-move sequences, and for each one, figure out whether it was a part of that 243 or not, and if not, what it was (and why). Here's what I found:

Sequences in Green = Repeats
Sequences in Yellow = Can be solved in 1 move
Sequences in Red = Can be solved in 0 moves
Sequences in Black = Unique cases that can be solved in 2 moves

F F (1*)	F F’	F B (2)	F B’	F L	F L’
F R	F R’	F U	F U’	F D	F D’
F M	F M’	F E	F E’	F S	F S’ (3)
F’ F	F’ F’ (1*)	F’ B	F’ B’ (4)	F’ L	F’ L’
F’ R	F’ R’	F’ U	F’ U’	F’ D	F’ D’
F’ M	F’ M’	F’ E	F’ E’	F’ S (5)	F’ S’
B F (2)	B F’	B B (6*)	B B’	B L	B L’
B R	B R’	B U	B U’	B D	B D’
B M	B M’	B E	B E’	B S (7)	B S’
B’ F	B’ F’ (4)	B’ B	B’ B’ (6*)	B’ L	B’ L’
B’ R	B’ R’	B’ U	B’ U’	B’ D	B’ D’
B’ M	B’ M’	B’ E	B’ E’	B’ S	B’ S’ (8)
L F	L F’	L B	L B’	L L (9*)	L L’
L R (10)	L R’	L U	L U’	L D	L D’
L M	L M’ (11)	L E	L E’	L S	L S’
L’ F	L’ F’	L’ B	L’ B’	L’ L	L’ L’ (9*)
L’ R	L’ R’ (12)	L’ U	L’ U’	L’ D	L’ D’
L’ M (13)	L’ M’	L’ E	L’ E’	L’ S	L’ S’
R F	R F’	R B	R B’	R L (10)	R L’
R R (14*)	R R’	R U	R U’	R D	R D’
R M (15)	R M’	R E	R E’	R S	R S’
R’ F	R’ F’	R’ B	R’ B’	R’ L	R’ L’ (12)
R’ R	R’ R’ (14*)	R’ U	R’ U’	R’ D	R’ D’
R’ M	R’ M’ (16)	R’ E	R’ E’	R’ S	R’ S’
U F	U F’	U B	U B’	U L	U L’
U R	U R’	U U (17*)	U U’	U D (18)	U D’
U M	U M’	U E (19)	U E’	U S	U S’

U’ F	U’ F’	U’ B	U’ B’	U’ L	U’ L’
U’ R	U’ R’	**U’ U**	**U’ U’** (17*)	**U’ D**	**U’ D’** (20)
U’ M	U’ M’	**U’ E**	**U’ E’** (21)	U’ S	U’ S’
D F	D F’	D B	D B’	D L	D L’
D R	D R’	**D U** (18)	**D U’**	**D D** (22*)	**D D’**
D M	D M’	**D E**	**D E’** (23)	D S	D S’
D’ F	D’ F’	D’ B	D’ B’	D’ L	D’ L’
D’ R	D’ R’	**D’ U**	**D’ U’** (20)	**D’ D**	**D’ D’** (22*)
D’ M	D’ M’	**D’ E** (24)	**D’ E’**	D’ S	D’ S’
M F	M F’	M B	M B’	**M L**	**M L’** (13)
M R (15)	**M R’**	M U	M U’	M D	M D’
M M (25*)	**M M’**	M E	M E’	M S	M S’
M’ F	M’ F’	M’ B	M’ B’	**M’ L** (11)	**M’ L’**
M’ R	**M’ R’** (16)	M’ U	M’ U’	M’ D	M’ D’
M’ M	**M’ M’** (25*)	M’ E	M’ E’	M’ S	M’ S’
E F	E F’	E B	E B’	E L	E L’
E R	E R’	**E U** (19)	**E U’**	**E D**	**E D’** (24)
E M	E M’	**E E** (26*)	**E E’**	E S	E S’
E’ F	E’ F’	E’ B	E’ B’	E’ L	E’ L’
E’ R	E’ R’	**E’ U**	**E’ U’** (21)	**E’ D** (23)	**E’ D’**
E’ M	E’ M’	**E’ E**	**E’ E’** (26*)	E’ S	E’ S’
S F	**S F’** (5)	**S B** (7)	**S B’**	S L	S L’
S R	S R’	S U	S U’	S D	S D’
S M	S M’	S E	S E’	**S S** (27*)	**S S’**
S’ F (3)	**S’ F’**	**S’ B**	**S’ B’** (8)	S’ L	S’ L’
S’ R	S’ R’	S’ U	S’ U’	S’ D	S’ D’
S’ M	S’ M’	S’ E	S’ E’	**S’ S**	**S’ S’** (27*)

Totals:
Black: 216 Unique cases that can be solved in 2 moves
Green: 54 Repeats (27 unique cases that each occur twice)
Yellow: 36 Cases that can be solved in 1 move
Red: 18 Cases that can be solved in 0 moves
216 + 27 = 243 possible unique cases that take 2 moves to solve when performing 2 moves

Figure 2.1.1 – A breakdown of the 324 total outcomes for 2-move sequences.

I'm defining what I call 'repeats' here as an outcome that takes the same amount of moves to solve as are in the sequence being evaluated (in this case, 2), and is an outcome more than once. I categorized these repeats separately from unique cases that can be solved in minimally 2 moves, even though all the repeats need at least 2 moves to solve as well. If you simply count all the cases that take at least 2 moves to solve, you'd end up with 270, which is not correct because its counting several states more than once. Amongst repeats, only the total amount of unique states within the repeats need to be counted. So if, for example, there were 100 total repeats, but only 5 unique states amongst those 100, then I'd only have to count those 5 toward the total count of positions (that wouldn't necessarily mean each of those 5 repeats 20 times, perhaps 4 of them repeat 10 times each and the 5th repeats 60 times). It turns out that when you separate the repeats, there was a total of 54 cases that repeated, 27 unique cases that each occur twice within the 324 total outcomes for 2-move sequences. When I add that 27 to the 216 unique cases I did find within the 324 total outcomes for 2-move sequences, I got 243, which is now, finally an accurate depiction of the unique count of positions that are exactly 2 moves away from being solved.

With all this in mind, the idea of what my program needs to count became clear. Starting with 18^0 all the way up to 18^{20}, for each x value, where x is the exponent of 18, my program will need to look at every possible sequence of moves, and decide whether the sequence results in a cube that can be solved in 0 moves, 1 move, 2 moves, and so on up to x moves, and if it can't be solved in any less than x moves, whether it's a unique state or if it's a repeat.

I began work on a program that can count each of these things by first deciding to represent a Rubik's Cube as a 54-integer array, where each index represented a specific cubie (Note: A cubie is the term that represents a single tile on a Rubik's Cube), and having numbers 1 through 6 represent the 6 different colors. Essentially, given the following template:

			0	1	2			
			3	4	5			
			6	7	8			
9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26
27	28	29	30	31	32	33	34	35
			36	37	38			
			39	40	41			
			42	43	44			
			45	46	47			
			48	49	50			
			51	52	53			

Figure 2.1.2 – A template that visually represents how a 3x3x3 Rubik's Cube will be stored as a 54 index array.

where that template represents a 3x3x3 Rubik's Cube two-dimensionally, and the numbers in each cubie of that template represent the indexes in an array, an array of a solved cube would look like this:

```
int[] solvedCube = {1,1,1,1,1,1,1,1,1,2,2,2,3,3,3,4,4,4,
                    2,2,2,3,3,3,4,4,4,2,2,2,3,3,3,4,4,4,
                    5,5,5,5,5,5,5,5,5,6,6,6,6,6,6,6,6,6};
```

Figure 2.1.3 – An array that represents a solved cube, based on Figure 2.1.2's template.

Next, I had to create a class that defined each of the 18 moves relative to this structure. As an example, the method I created to perform a left face turn looks like this:

```
/*Performs a left slice clockwise quarter turn*/
public void left(int[] array){
	int temp = array[0];
	array[0] = array[45];
	array[45] = array[36];
	array[36] = array[12];
	array[12] = temp;

	temp = array[3];
	array[3] = array[48];
	array[48] = array[39];
	array[39] = array[21];
	array[21] = temp;

	temp = array[6];
	array[6] = array[51];
	array[51] = array[42];
	array[42] = array[30];
	array[30] = temp;

	temp = array[9];
	array[9] = array[27];
	array[27] = array[29];
	array[29] = array[11];
	array[11] = temp;

	temp = array[10];
	array[10] = array[18];
	array[18] = array[28];
	array[28] = array[20];
	array[20] = temp;
}
```

Figure 2.1.4 – A method that performs a left face turn on the cube while it's being represented by an array.

In that method, there are 5 groupings of position swaps that occur in cycles. The faces edges' must cycle, as do its corners, but then the cubies that aren't actually on the face being moved that still get effected must be updated as well, which can be grouped as left corners, edges, and right corners.

I ultimately created a method for each of the 18 moves, as well as methods that rotate the entire cube about the x, y, and z-axis's respectively, and finally an orientation method that sets the cube in a specific position for comparison purposes by utilizing the aforementioned x/y/z-axis rotation methods. Orientation is possible because the center pieces are stationary,

and it is necessary because otherwise two equivalent cases could be compared but not found to be equal if one is oriented differently than the other. A simple example of this would be if I performed a sequence like, LR', and compared it to a sequence of M'. If the orientation didn't occur, they would not be found to be equal, since the M' sequence needs to be reoriented by being rotated counterclockwise about the Y axis before being compared for equivalence.

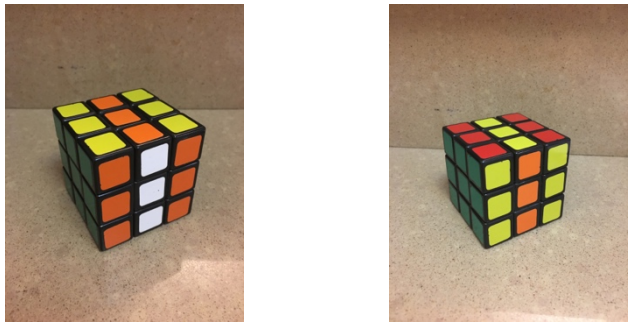


Figure 2.1.5 – In case 1 the move M' was performed, while in case 2, the moves LR' were performed. In the first image, the cube needs to be reoriented, while in the second image, it already is oriented correctly. Without reorientation, despite these two cubes being the same, they wouldn't be found to be a match.

Once I wrote the class that defines the cubes orientation and movements, the logic was pretty straightforward. I created an ArrayList of 21 binary search trees, which each exist to store cube states at each distance, 0 through 20. Starting at distance 0 and moving my way up, I fill each tree with the appropriate cube states, so they can be used to compare at larger distances. The reason I used a binary search tree to store states at each distance is that when attempting to find a match, instead of having to compare the cube state I'm searching for to every single previously stored state at each distance, I only have to compare it to log(n) of those amounts at each distance. As the amounts of states at each distance get quite large as x grows, it makes a *huge* difference. As an example, at only distance 7, there are already 100,803,036 states to be stored according to the chart from cube20.org, in figure 1.5.1. Instead of doing that many comparisons, if I were using the HTM (my values may end up being different since I'm using the QTSM), I would only have to do $\log_2(100{,}803{,}036)$ comparisons, which is approximately 26.58. Even with rounding that up to 27, it is obviously *much* better having search run at O(log(n)) than at O(n). The tree that stores distance-0 cube states only has 1 state in it, and the tree that stores distance-1 cube states has 18 states in it. For distance-x and above, where x is larger than or equal to 2, for each possible state at a given distance-x, I do a search for the current cube state in question in each of the trees that are storing cube states that are a distance that is less than or equal to x. The reason I search for the current cube state in the tree storing the current x-value states is because if it is found to already exist in that tree, then that means it is a repeat, and mustn't be added to the tree again. Finally, after all of the searches in each of the trees storing states less than or equal to distance-x, if no match has been found, the cube state is added to the current distance-x tree. If a match is found, on the other hand, then the proper counter is incremented appropriately. Finally, after every possible sequence at distance-x has been exhausted, the results are output, the counters are all reset, x is incremented by 1 and the process begins again.

2.2 – 2x2x2 Rubik's Cube

Given the amount of calculations necessary for a program of this nature, I thought it would make sense to write a similar program that does the same thing for a 2x2x2 Rubik's Cube first, as a proof of concept that this strategy can work, given enough storage space, computational power, and time. I also thought it made sense to run this analysis on the 2x2x2 as well because the data might be useful in building a general formula for an NxNxN cube when compared to the data for a 3x3x3 cube.

In order to accomplish this, a few things had to be changed from my original program. Firstly, the orientation methods had to be modified to apply to a 2x2x2 cube, which would be represented as a 24 index array, instead of a 54 index array like the 3x3x3. Secondly, all of the mid-slice methods were deleted, since a 2x2x2 only has faces, no mid-slices. Finally, a new re-orient method had to be written. Since 2x2x2's don't have center cubies, which are stationary on the 3x3x3, and thus can easily be used to re-orient a 3x3x3, the 2x2x2 had to be oriented to 1 corner being in a certain position and orientation all the time. Ultimately, the run time on my 2x2x2 program should be a lot faster than that of my 3x3x3 program, given that the 2x2x2 only has 3,674,160 unique states, compared to the 3x3x3's 43,252,003,274,489,856,000. This figure is represented as

$$\frac{8! * 3^7}{24} = 3{,}674{,}160$$

Figure 2.2.1 – Equation for total states possible on a 2x2x2 Rubik's Cube.

where 8! is the total amount of permutations of the 8 corners, and 3^7 represents all the possible orientations of those 8 corners, as each corner has 3 sides. Like the 3x3x3, only 7 of the corners can be oriented independently, and the orientation of the eighth depends on the orientation of the preceding seven. It is divided by 24 because there is nothing identifying the orientation of the cube in space. This is because all 24 possible positions and orientations of the first corner are equivalent due to the lack of fixed centers [13].

Beyond that, the 2x2x2 works the same way as the 3x3x3, except that it only has to go up to distance-14, since it has already been proven that God's Number for the 2x2x2 is 14 for the Quarter Turn Metric (QTM), which on the 2x2x2 also happens to be the same as the Quarter Turn Slice Metric (QTSM), since the 2x2x2 doesn't have any middle slices (For the Half Turn Metric, God's Number is 11) [14].

2.3 – Problems with My Initial Approach

I knew from the outset that I'd require the use of a super computer in order to even attempt running my program, but once I began working on it in earnest, I quickly realized that even a super computer would likely not be powerful enough for me to complete it. There's a reason researchers used symmetry and mirroring to cut down the amount of cases they needed to solve in order to prove God's Number; 43,252,004,274,489,856,000 is a really, *really* big number. My program, by design, is meant to go through (13,498,132,464,419,376,890,580,751, which is approx. 3.1 * 10^6 times larger than the total amount of cube states on a 3x3x3) sequences, which is obviously extremely inefficient and problematic. Even beyond that, each of those sequences needs to do $\log_2(n)$ comparisons for each tree less than or equal to the current x-value of the sequence in question, where n is the number of states stored in the tree being searched.

Even if I had a super computer powerful enough to chug through all of those sequences, there is still the problem of storage. My program is representing single cube states as a BigInteger that is 54 digits long, with each BigInteger taking up 280 bytes (BigIntegers are advanced data types, represented as objects, and as such store large numbers as arrays of single integers, plus additional overhead like padding, sign value and length). Even in an optimized scenario, where each state could be stored with a single byte, given that I'm trying to store over 43 quintillion states, I'd need approximately 43 million terabytes of storage space to store them all:

$$1 \text{ terabyte } = 10^{12} \text{ bytes}$$

$$\frac{8! * 3^7 * \left(\frac{12!}{2}\right) * 2^{11}}{10^{12}} \approx 4.3 * 10^7$$

Figure 2.3.1 – A calculation of approximately how much storage would be necessary to successfully run my program.

Given that even with the super computer it likely wouldn't be possible to actually run my program to completion, I decided that I would run the programs for as long as I could and collect as much data as I could mine. This approach was born out of the idea that this data would provide me the means to build a mathematical formula that describes how the totals at each distance grow (or at the very least, clues as to how to proceed with this problem), so I always planned on leaning on mathematics to help me get closer to solving this problem anyway. The hope of actually proving God's Number this way was always a bit of a reach; I just knew that if I actually execute a program of this manner to completion that it would definitively do so. Proving God's Number solely through a program of this nature, without any mathematical component to complement it would go against the spirit of what I'm trying to accomplish here in the first place. In essence I would be brute forcing an answer, in a similar, but less efficient way to the researchers who solved God's Number for the Half Turn Metric.

The data I find, however, should be useful in helping me discover patterns that might help me build a function for the total amount of cases that are distance-x away from being solved.

Chapter 3 – Results to My Initial Approach

3.1 – Results of the 2x2x2 Program

After completing my 2x2x2 program, here is what I found:

		When there are x amount of moves in a sequence…				
		0	1	2	3	4
Amount of cases that take y moves to solve.	0	1	0	24	0	1152
	1	x	12	0	576	0
	2	x	x	120	0	8448
	3	x	x	x	1152	0
	4	x	x	x	x	11136
	5	x	x	x	x	x
	6	x	x	x	x	x
	7	x	x	x	x	x
	8	x	x	x	x	x
	9	x	x	x	x	x
	10	x	x	x	x	x
	11	x	x	x	x	x
	12	x	x	x	x	x
	13	x	x	x	x	x
	14	x	x	x	x	x
Repeats:		0	6	93	1032	10602
Uniques:		1	6	27	120	534
Total:		1	12	144	1728	20736

When there are x amount of moves in a sequence

		5	6	7	8	9
Amount of cases that take y moves to solve.	0	0	67584	0	4374528	0
	1	33792	0	2187264	0	150331392
	2	0	586752	0	41914368	0
	3	107520	0	8706048	0	688570368
	4	0	1308672	0	122732544	0
	5	107520	0	15470592	0	1673650176
	6	X	1022976	0	177463296	0
	7	X	x	9467904	0	1934032896
	8	X	x	x	83496960	0
	9	X	x	x	X	713195520
	10	X	x	x	X	x
	11	X	x	x	X	x
	12	X	x	x	X	x
	13	X	x	x	X	x
	14	X	x	x	X	x
Repeats:		105264	1014007	9434846	83382811	712835012
Uniques:		2256	8969	33058	114149	360508
Total:		248832	2985984	35831808	429981696	5159780352

When there are x amount of moves in a sequence…

		10	11	12	13	14
Amount of cases that take y moves to solve.	0	300662784	0			
	1	0	10777264128			
	2	3082223616	0			
	3	0	54672457728			
	4	10815553536	0			
	5	0	164831232000			
	6	22057844736	0			
	7	0	272987848704			
	8	19904888832	0			
	9	0	198771081216			
	10	5756190720	0			
	11	x	40968486912			
	12	x	x			
	13	x	x	x		
	14	x	x	x	x	
Repeats:		5755260132	40967136060			
Uniques:		930588	1350852			
Total:		61917364224	743008370688			

Figure 3.1.1 – Chart displaying the output of my 2x2x2 program.

While I wasn't yet able to complete the execution of my 2x2x2 program, I did manage to get enough results that I could populate the chart up to the x-value of 11. I had been working on parallelizing my programs and setting it up so multiple computers could perform searches for matches simultaneously, but time constraints didn't allow me to complete that step. However, the data I did find was immediately useful. When checking the data from the proof that already exists for God's Number on the 2x2x2 using the QTM, I found that my count for unique states at each distance matched up perfectly with theirs.

```
     Analysis of the 2x2x2 cube group
        --------------------------------

Originally computed on a DEC VAX 11/780 in over 51 hours of
CPU time on Sept. 9, 1981
          Moves Deep              arrangements (q)

                0                           1
                1                           6
                2                          27
                3                         120
                4                         534
                5                       2,256
                6                       8,969
                7                      33,058
                8                     114,149
                9                     360,508
                10                    930,588
                11                  1,350,852
                12                    782,536
                13                     90,280
                14                        276
                                      -------
                                    3,674,160
```

Figure 3.1.2 – An excerpt from http://cubeman.org/fullcube.txt that displays the total amount of cases that exist at each distance from solved on the 2x2x2 Rubik's Cube. This is the work of Dan Hoey, Jerry Bryan, Dik Winter, Micheal Reid, Martin Schoenert, Tom Rokicki, Jaap Scherphuis and Mark Longridge [18].

The chart in figure 3.1.2, when compared to the 'Unique' amounts from my results, show that my program was working correctly, as it was finding the same final results. The difference, however, is that my program also collected the data for the rest of the sequences at each distance, with the hopes that some pattern might emerge.

While there are a few things I want to comment on when it comes to these results, the thing I first want to discuss is how every other result is 0. At first I thought there might be some type of error in my code, but then I realized that the totals were still accurate, so the 0's must be too. What I realized is, because there are only 2 layers, its not possible to solve a cube in an odd amount of moves less than the length of the sequence. On a 3x3x3 Rubik's Cube, if you have a 2-move sequence of LR', it can be solved in 1 move (by performing M). This is only possible because of the existence of a 3rd layer, which allows for sequences of parallel moves that aren't the equivalent to an entire cube rotation, like they would be on the 2x2x2. If I were to perform the same LR' sequence on a 2x2x2, it would result in a cube that takes 0 moves to solve, because that sequence on a 2x2x2 is essentially just a rotation about the x-axis. On a 3x3x3, that was the only situation I was able to see that allowed a sequence to get solved in 1

less move than the length of the sequence, but I hadn't accounted for that causing the entire 2x2x2 Rubik's Cube to only be able to be solved with solutions that share an evenness with the original sequence. That is to say, if a sequences' length is odd, a 2x2x2 cube can only possibly be optimally solved in an odd number of moves that is less than or equal to the original sequences length. It is also true that if a sequences length is even, a 2x2x2 cube can only be optimally solved by an even number of moves (including 0) that is less than or equal to the original sequences length.

Ultimately, while the importance of parallelism became clear to me due to the occurrence of 0's in my 2x2x2 programs' results, because of them, it'll likely be tougher to build a formula (once I eventually complete the execution of my 2x2x2 program and finish filling in my chart) that is easily generalized to work for any sized Rubik's Cube.

3.2 – Findings of My Initial Approach

After completing my 3x3x3 program, here is what I found:

When there are x amount of moves..

		0	1	2	3	4	5	6	7
Amount	0	1	0	18	36	720	3600	42624	
of cases	1	x	18	36	720	3600	42624	312480	
that take	2	x	x	270	1188	18360	130032	1428168	
y moves	3	x	x	x	3888	25920	392592	3403824	
to solve	4	x	x	x	x	56376	510624	7935120	
	5	x	x	x	x	x	810096	9332880	
	6	x	x	x	x	x	x	11557128	
	7	x	x	x	x	x	x	x	
	8	x	x	x	x	x	x	x	x
	9	x	x	x	x	x	x	x	x
	10	x	x	x	x	x	x	x	x
	11	x	x	x	x	x	x	x	x
	12	x	x	x	x	x	x	x	x
	13	x	x	x	x	x	x	x	x
	14	x	x	x	x	x	x	x	x
	15	x	x	x	x	x	x	x	x
	16	x	x	x	x	x	x	x	x
	17	x	x	x	x	x	x	x	x
	18	x	x	x	x	x	x	x	x
	19	x	x	x	x	x	x	x	x
	20	x	x	x	x	x	x	x	x
Repeats:		0	0	27	648	13561	251166	4322847	
Uniques:		1	18	243	3240	42815	558930	7234281	
Total:		1	18	324	5832	104976	1889568	34012224	

Figure 3.2.1 – Chart displaying the output of my 3x3x3 program. I didn't show x-values 8 through 20 because I was only able to collect data up to distance 6.

I was only able to get to distance 6 because at distance 6, I was already storing a tree with 7 million plus nodes, as well as the trees with less nodes in memory and ran into the following error:

```
Exception in thread "main" java.lang.OutOfMemoryError: GC overhead limit exceeded
        at java.math.BigInteger.add(BigInteger.java:1315)
        at java.math.BigInteger.add(BigInteger.java:1221)
        at cube.ThreeByThree.convert(ThreeByThree.java:293)
        at cube.ThreeByThree.seven(ThreeByThree.java:573)
        at cube.ThreeByThree.run(ThreeByThree.java:1718)
        at cube.ThreeByThree.main(ThreeByThree.java:1735)
```

Figure 3.2.2 – An OutOfMemoryError that occurred from storing too much in memory.

In order to resolve this situation, I need to reconfigure my 3x3x3 code so it stores the trees either directly to the hard drive, or in a database. I didn't run into this issue with my 2x2x2 code because in total there are only a little more than 3 million possible cube states, and therefore the same amount of nodes.

As far as the actual data is concerned, there were several things that stood out to me after finally getting this data from my program. The first thing I noticed, of course, is that unlike the 2x2x2 program's results, there are essentially no zeroes in the data (with the exception of cases that could be solved in 0 moves after performing 1 move to get to the states in question, which makes sense because if you're only performing 1 move, of course you'll only be able to solve it by doing that original moves prime, and never by performing 0 moves). This makes sense because of the existence of the 3rd layer. When looking at the data for the first couple x values, you can kind of make sense of certain things that are happening. For example, in the breakdown of distance 2, it makes sense that there are 18 cases where the result can be solved in 0 moves. The only 2-move sequences that can possibly result in a solved cube are the sequences when the 2nd move is the prime of the first move, and since there are 18 possible moves on a 3x3x3 in the QTSM, it follows that there would only be 18 cases where the result can be solved in 0 moves. It also makes sense that there are 36 cases that can be solved in 1 move at distance 2. As we learned from the results of the 2x2x2 program, the only situations where a cube can be solved in 1 less move than the length of the sequence (or any odd amount less than the sequences length, for that matter), are when the sequence has 2 moves that are parallel. It can be surmised that every move has 2 parallel moves. As an example, if you performed a L on a cube, its parallel moves are M and R', meaning if you have a 2 move sequence of LM, it can be solved in only 1 move, by doing R'.

Figure 3.2.3 – The sequence shows how a L and R' sequence results in an M' sequence (after being reoriented), and can be solved with a 1 move sequence, in this case, M.

Another interesting aspect of my results was the fact that for any given distance x, the amount of cases that took 0 moves to solve was equal to the amount of cases that took 1 move to solve at distance x - 1. I can't say for certain that this trend would continue, as I was only able to collect data up to a certain point, but it does make sense that the values would be connected in this way. At distance 3, for example, since there were 36 cases at distance 2 that could be solved in 1 move, if you perform any of the 18 moves after each of those sequences, we know that 1 of those moves will be the 1 move needed to solve the cube. If you think of the possibilities in a combinatory way, you'll get the following:

$$18 * 2 * 1 = 36$$

Figure 3.2.4 – In the above, 18 represents the first move, which can be anything. The 2 represents the 2nd move, which can only be 1 of 2 possible moves, because it needs to be a parallel move in order to result in a 2 move sequence that can be solved in 1 move. The 1 represents the 3rd move, which can only be the other parallel move, which solves the cube.

The more I poured over these results, the more I began to realize the importance of the concept that these numbers are based on the fact that there are different *kinds* of moves, relative to a first move.

Chapter 4 – Next Approach

4.1 – Expansion into Mathematical Analysis

Once I realized how important the fact that different kinds of moves exist was, I knew that the data had served its purpose in guiding me into figuring out the next step towards finding God's Number for the 3x3x3 using the QTSM, and possibly finding a general formula for God's Number for any NxNxN sized cube using the QTSM.

As far as I can tell, there are 5 different types of moves. Given a random starting move, the next move in a sequence can be any of the following:

1) The starting moves' prime
2) The starting move, repeated
3) 1 of the 2 parallel moves of the starting move
4) 1 of the 2 non-intersecting, non-parallel moves of the starting move
5) One of the other 12 intersecting moves

As an example, if we look at L as the first move, we would get the following in each category:

1) L'
2) L
3) M and R'
4) M' and R
5) B, B', F, F', S, S', U, U', D, D', E, E'

For clarities sake, when I say parallel, I mean a move that is about the same axis as the first move, as well as in the same direction as the first move. So if the first move was about the x-axis and was clockwise, for the 2nd move to be parallel, it also needs to be about the x-axis and be clockwise, but it can't be a repeat of the first move, as that is its own, separate kind of move. When I say non-intersecting, non-parallel move, all I mean is that the 2nd move is about the same axis as the first, but turns in the opposite direction as the first. So if the first move was about the y-axis and counterclockwise, for the 2nd move to be non-intersecting and non-parallel, it would also need to be about the y-axis, but would need to turn clockwise instead.

Using this framework to re-look at the analysis of 2 move sequences from figure 2.1.1, all 324 cases can be explained more effectively. The 18 cases that are able to be solved in 0 moves are all the cases where the 2nd move is the first moves' prime, or written as a combination, 18 * 1. The 36 cases that can be solved in 1 move are cases where the second move is any of the first moves' parallel moves, or written as a combination, 18 * 2.

4.1.1 – Repeats

The 27 repeats are a little more complicated, but still can be explained easily enough. If you look back at figure 2.1.1 again, you'll see that I put an asterisk next to some of the repeat cases. The reason I did this is to differentiate between the two different ways in which I've been able to surmise that repeats occur. The cases with an asterisk are the 18 cases of a move followed by the same move again, or as a combination, 18 * 1. This creates repeats because, as an example, FF is two 90° turns, or 180°. Since a slice can get to 180° by going 2 repeating moves in either direction, the result occurs twice within the 324 total sequences; once by performing FF, and again by performing F'F'. At distance 2, there are 18 cases where a move is followed by the same move again. Since each outcome pairs with another outcome, there are 9 unique results within those 18 cases, and 9 repeats. The other 36 cases that result in a case that is repeated are cases when a non-intersecting, non-parallel move occur after the first move, or as a combination, 18 * 2. The reason this would create a repeat is more straightforward than the reasoning behind the other repeats I just explained.

Since the two moves don't intersect, the sequence has a commutative property, meaning you can arrive at the same result by performing the sequence in any order. Historically, the cube has been thought of as not being communicative, but I would argue that it actually is *partially* commutative. What I mean by this is, while it is generally not commutative, given a sequence of non-intersecting moves (parallel or non-parallel), it will maintain its commutative property. This is important because besides a move occurring twice in a row, it is the only way I know of to cause repeats to occur.

Of course, if you were attempting to calculate all the repeats that occur within a sequence longer than 2 moves, there will be sub-sequences within some sequences that cause repeats as well. An example of this would be the sequence RUD. R and U are intersecting moves, but U and D aren't, nor are they parallel. The means that the result of the sequence RUD can also be arrived at by performing the sequence RDU, because of the partial commutative property I mentioned earlier.

All together, if you take the 9 repeat states that result from the 18 cases where a move is followed by itself, and the 18 repeat cases that result from the 36 cases where a move followed by a non-intersecting, non-parallel move, you get 27 total repeat cases, and 27 unique cases. The other 216 unique cases all occur when you perform a move, and follow it with an intersecting move, which when written as a combination is 18 * 12. Using the categories and combinations to view the totals looks like this:

Category	Combination	Can be solved in	Repeats	Total
1) The starting moves prime:	18 * 1	0	9	18
2) The starting move, repeated:	18 * 1	2	9	18
3) A parallel move	18 * 2	1	18	36
4) Non-intersecting/non-parallel	18 * 2	2	18	36
5) Intersecting move	18 * 12	2	0	216

Figure 4.1.1 – This chart shows that all 324 moves are easily accounted for when factoring in move type. An interesting side effect of this is the knowledge that repeats occur at distances less than the current x value as well, which can be useful later on when attempting to derive a formula that summarizes this breakdown for all x values.

4.2 - How to Next Approach the God's Number Problem

With all this in mind as far as the kinds of moves go, and how it is relatively straightforward to do a breakdown of the amount of cases at each distance, my next idea for filling the rest of my chart has formed. If I were to build a data tree, where the root is represented by the first move, and each of its 5 children are represented by a different kind of move, and let it grow to a depth of 20, hypothetically anyway, I'd be able to calculate everything necessary to complete my chart and prove God's Number using the QTSM.

Of course, its not at all that simple, or else I'd have built it and calculated it already, but having the starting point is at least progress. The first reason I can come up with as to why its not that simple is the fact that there are certain rules that would have to be accounted for when building such a tree. To begin with, lets look at what that tree might look like with a height of 2:

```
                    18
/       /           |           \       \
12      2R          2(-1)       1R      1(-2)
```

Figure 4.2.1 – What a data tree of my design would look like with a height of 2.

In the tree above, the R's represent kinds of moves that cause repeats to occur. To calculate the totals, you'd have to multiply each node on a branch, up to the root, and know what a node means. In this case you'd first have 18 * 12, or 216, which represents a random first move, and an intersecting move, for the first branch. Next you'd have 18 * 2R, which represents a random first move, and 1 of the 2 non-intersecting, non-parallel moves, which gives you an outcome of 36, but because 1 R appears in the branch, you divide by 2. In a given branch, whatever your total is, you'd have to divide by 2 every time an R appears in that branch. Therefore, the second branch would result in 18. The third branch would be 18 * 2(-1), which represents a random first move, and a parallel move. The -1 you see in the tree means that every time you see a -1 in a branch, you'd have to subtract 1 from the length of the branch in order to know how many moves it would take to solve a particular sequence that fits in that branch. The fourth branch would be 18 * 1R, or 18, which is representing a random move,

followed by that same move a 2nd time. As the R appears in this branch, you have to divide by 2 once and you end up with 9. Finally, the last branch would be 18 * 1(-2), which represents a random first move, and that moves prime. The -2 means you need to subtract 2 from the length of the branch in order to know how many moves it would take to solve a particular sequence that fits in that branch. In this branches case, that means 18 cases can be solved in 0 moves.

If you sum up everything at the 2nd level, you get 216 cases that can be solved in 2 moves in the first branch, 18 cases that can be solved in 2 moves in the 2nd branch, 36 cases that can be solved in 1 move in the 3rd branch, 9 cases that can be solved in 2 moves in the 4th branch, and 18 cases that can be solved in 0 moves in the 5th branch. When you sum that up, you get 243 cases that can be solved in 2 moves, 36 cases that can be solved in 1 move, and 18 cases that can be solved in 0 moves. You can also calculate the number of repeats by taking 18 raised to the xth power, where x is the height of the tree, and subtracting the sum of your results from it. In this case that would be:

$$324 - (216 + 18 + 9 + 36 + 18) = 27$$

Figure 4.2.2 – In the equation, 324 is 18^2, 216, 18, and 9 are the unique cases that can be solved in 2 moves, 36 is the amount of cases that can be solved in 1 move, and 18 is the amount of cases that can be solved in 0 moves. The result, 27, is the amount of repeats.

The difficulty begins at the 3rd level, when you have to consider how the different kinds of moves interact with one another. For example, if you have a branch that follows the sequence 18 – 1(-2) – 1(-2), what happens? Its only a 3 move sequence, so you can't subtract 4, so right off the bat we know something different has to happen. If we remember from earlier, the 1(-2) represented the prime of the previous move, so a sequence of 18 – 1(-2) – 1(-2) would mean that a random first move occurs, the 2nd move is the first moves prime, thus cancelling the first move out and leaving you with a solved cube, but then the 3rd move is the prime of the 2nd move, leaving you back where you were after completing 1 move in the sequence.

Another example is the sequence 18 – 12 – 1(-2) – 12, where the 4th move is the 1st moves prime. Initially, its clear to see that the 2nd and 3rd moves cancel one another out, but in this instance, one of the 12 possible moves that was picked 4th happened to be the 1st moves prime, and because the 2nd and 3rd cancelled each other out, its possible for the 1st and 4th to cancel each other out as well, so you end up with a sequence that takes 0 moves to solve.

The point is while not impossible, there is a lot of underlying logic that would have to be figured out before I could even attempt to calculate the rest of my chart based on this type of layout. The reason I haven't yet tried to do this is because of the more significant problem with this approach.

Presume for a moment you're looking at the branch all the way on the left, 18 – 12 – 12 – 12…. and so on. It turns out there is a point where that branch is larger than the total amount of cube states in a 3x3x3 Rubik's Cube, by itself. At a height of 19,

$$18 * 12^{18} > 8! * 3^7 * \left(\frac{12!}{2}\right) * 2^{11}$$

Figure 4.2.3 – In the equation, $18 * 12^{18}$ represents a branch at depth 19 accounting for all sequences where every move after the first represents 1 of the 12 moves that intersects the previous move.

While it *is* possible that God's Number for the QTSM is only 18 (in fact, if that were the case, this would actually be a really good partial explanation for why), the fact of the matter is sequences of that nature necessarily must be able to be solved in less moves than the length of the sequence in question. This *has* to be the case because we know there are a finite amount of states, and therefore a God's Number exists. As such, if you follow that pattern continuously, eventually you'll end up with a result that takes less moves to solve than the length of the sequence in question (once the length of the sequence in question is larger than God's Number). While I initially can't come up with any particular sequence that would follow that branch at any depth, I do know about the existence of cycles, which could possibly fall into that category and explain how sequences with only intersecting moves eventually can be solved in less moves than the length of their sequence.

4.3 – Cycles

In the study of mathematics related to the Rubik's Cube, a popular topic is cycles. Cycles are sets of cubies that swap positions with one another based on moves. Typically, more than 1 swap is going on at once, as I discussed in chapter 1.2 when describing what happens when a single cube face is rotated, but many times, a sequence is utilized that will cycle 3 cubies.

More importantly for our current conversation, cycles are relevant because it has been shown that any sequence, if repeated enough times, will bring a cube back to its original state. The existence of such a way to arrive at a solved state, beyond utilizing the 5 types of moves I outlined in chapter 4.1, means that in order to build the tree that would populate the rest of the data necessary to complete the chart in figure 3.2.1, I'd have to account for the possibility that cycles could alter my results from building such a tree.

As a first pass at attempting to analyze cycles, I wrote a program that repeats all possible sequences at every length of sequence from 0 to 10, and tests against the solved cube until the total amount of moves performed was 20. The idea behind that was that if a cycle needs to repeat enough times that the total amount of moves is more than 20, the cycle in question won't be relevant to my calculations. I initially didn't think there would be that many cycles that resulted from a total amount of moves that was less than or equal to 20. My program took approximately 16 days to execute to completion, and resulted in approximately

143.2 GB of text files listing all of the cycles, so saying I was wrong about that would definitely be an understatement.

Chapter 5 - Conclusion

5.1 – Results

In the end, this project is still very much a work in progress, though I definitely think that considering the fact that I'm tackling a problem that took researchers 30 years to solve (using a different metric of course), I've made a fairly impressive amount of progress. While it was never truly realistic for my 3x3x3 program to finish running, the data that it did uncover managed to be really helpful in helping me navigate to the next step in breaking down this problem.

God's Number for the QTSM is still a mystery, but I'd like to think that if I am able to fully populate the chart from figure 3.2.1, I'll be able to write an equation of the following form:

$$\sum_{x=0}^{G} 18^x - L - R = 8! * 3^7 * \left(\frac{12!}{2}\right) * 2^{11}$$

Figure 5.1.1 – In the equation, G represents God's Number, L represents the total number of states that exist with a sequence of x moves which can be solved in less moves than the value of x, R represents the total number of states that are repeats at distance x, and $8! * 3^7 * \left(\frac{12!}{2}\right) * 2^{11}$ represents the total number of cube states in a 3x3x3 Rubik's Cube.

Or, more generally:

$$\sum_{x=0}^{G} 6n^x - L - R = \frac{8! * 3^7}{24^{(n+1) mod\, 2}} * (12! * 2^{10})^{n\, mod\, 2} * 24!^{\left\lfloor \frac{n-2}{2} \right\rfloor} * \left(\frac{24!}{4!^6}\right)^{\left\lfloor \left(\frac{n-2}{2}\right)^2 \right\rfloor}$$

Figure 5.1.2 – In the equation, G represents God's Number, L represents the total number of states that exist with a sequence of x moves which can be solved in less moves than the value of x, R represents the total number of states that are repeats at distance x, and $\frac{8! * 3^7}{24^{(n+1) mod\, 2}} * (12! * 2^{10})^{n\, mod\, 2} * 24!^{\left\lfloor \frac{n-2}{2} \right\rfloor} * \left(\frac{24!}{4!^6}\right)^{\left\lfloor \left(\frac{n-2}{2}\right)^2 \right\rfloor}$ is the formula that outputs the total number of possible cube states for a given n cube size.

Where given an input n, where n is the cube size, when the formula in figure 5.1.2 is a proper equivalence, the value of G will be God's Number for that n value.

In order to do so, I still need to either prove that cycles of the form 18 – 12 – 12 – 12... don't exist at total move lengths of less than God's Number, or if they do, account for them in the above equation, as well as work out the logic necessary to build the tree I described earlier that could properly account for the rest of the data necessary to fill the chart in figure 3.2.1, then actually write a program that builds that tree, along with the proper logic, and run it to completion. What's nice to know is that as described, a tree of that nature would be more efficient at deriving the necessary data by several orders of magnitude. As each node would

have 5 children, at depth 20, there would be 5^{19} branches to calculate in order to collect all the necessary data. 5^{19} is approximately 19.7 trillion, which means that while its still a lot of calculations, its still a realistic amount for even a regular desktop computer to compute.

Following that logic, it wouldn't be that much more difficult to build a tree of this nature for a 4x4x4 Rubik's Cube, or to use the the data derived from the 3x3x3 tree to build a general formula that outputs the total number of unique states that take x moves to solve a cube at distance x, for any sized cube. With such a general formula, I'd be able to add it to the program I wrote that calculates the total number of cube states for an NxNxN cube, where the user inputs the N value, and test for correctness. With enough God's Numbers' calculated at enough N values, its likely that a formula that describes God's Number at a given N value would be able to be derived.

5.2 – Potential Applications of my Findings

Right off the bat, by describing a new method to discover God's Number, the most obvious application I can think of is discovering God's Number for the QTSM, as well as other metrics. It wouldn't take too much modification for such a tree to be built utilizing a different metric.

Beyond that, once God's Number is known, fully, I imagine plenty of potential applications to use it advantageously. I have an idea for a potential compression algorithm that would utilize the 54 index array in a similar way to how a computers' bus takes a bit string and derives instructions from it. In this potential algorithm, you'd be able to take a cube state, have it give you a piece of data, like a color, as well as instructions for a sequence of moves to bring you to the next cube state, which in turn tells you another color and more instructions. It would continue this way until enough colors have been laid out so as to depict an image. Essentially, I'd be able to take an image, and using the 43 quintillion+ amount of unique cube states, starting with the last pixel, work backwards to find what state I need to start with in order to reconstruct the image from scratch. Since God's Number is no larger than 20, that would mean that at most a program would need to do O(1) operations per pixel to decompress an image that, as this idea describes, can be stored in a single 54 index array, taking up only a couple hundred bytes. What I really love about this idea is that, even if, somehow, 43 quintillion unique pieces of information is somehow not specific enough to properly build instructions to compress/decompress an image, or other piece of data, to a couple hundred bytes, all you'd have to do is upgrade from a 3x3x3 to a 4x4x4, or 5x5x5, or whatever you ultimately need to fully cover the complete specificity necessary to properly be able to compress and decompress a file.

Works Cited

1. http://cube20.org
2. https://docs.google.com/file/d/0Bx0h7-zg0f4NTUM1MWUtaDJrbms/edit?pli=1
3. http://arxiv.org/pdf/1106.5736v1.pdf
4. http://librarydb.saintpeters.edu:8080/bitstream/123456789/198/1/Hollingsworth_Nykosi.pdf
5. http://kociemba.org/cube.htm
6. http://www.mementoslangues.fr/CubeDesign/3xCubes/RubiksCubeSolutions.pdf
7. http://www.telegraph.co.uk/technology/google/10840482/Rubiks-cube-invention-40-years-old-and-never-meant-to-be-a-toy.html
8. https://www.rubiks.com/about/the-history-of-the-rubiks-cube
9. https://www.speedsolving.com/wiki/index.php/Metric
10. http://www.jaapsch.net/puzzles/thistle.htm
11. http://www.jaapsch.net/puzzles/compcube.htm#kocal
12. https://en.wikipedia.org/wiki/Rubik%27s_Cube#Permutations
13. https://en.wikipedia.org/wiki/Pocket_Cube
14. http://www.jaapsch.net/puzzles/cube2.htm
15. http://www.ryanheise.com/cube/cube_laws.html
16. http://www.ryanheise.com/cube/parity.html
17. http://tomas.rokicki.com/rubik20.pdf
18. http://cubeman.org/fullcube.txt

Appendix A

ThreeByThreeCube

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: ThreeByThreeCube
 * Class: ThreeByThree
 * Purpose of this Program: The purpose of this program is to systematically
 *                go through every possible case a 3x3x3 Rubik's cube can exist in,
 *                in order to be able to say the following:
 *                "When there are x moves performed on a 3x3x3 Rubik's Cube,
 *                there are ____ cases that take y moves to solve."
 *                A chart that will display the results is set up with incrementing x values
 *                 in each column, incrementing y values for each row, and the blank will be
 *                filled in with the correct value to appropriately complete the statement.
 *  Purpose of this Class: This class runs the main program which works like this:
 *                Starting with x = 0, all the way up to x = 20, the program will perform all
 *                possible sequences of x moves, then try to find matches amongst all states that
 *                have already been stored, and if so, will count it accordingly.
 *                For example, When x = 3, the program will compute every outcome that results
 *                from performing 3 moves on the 3x3x3 Rubik's Cube. Each outcome will go
 *                through the following:
 *                1) It will be traverse the binary search tree that stores the 1 state that
 *                        takes 0 moves to solve, and if a match is found, it will increment the
 *                        zeroCounter.
 *                        2) It will traverse the binary search tree that stores all 18 states that
 *                        take 1 move to solve, and if a match is found, it will increment the
                oneCounter.
 *                3) It will traverse the binary search tree that stores all 243 states that take 2
 *                          moves to solve, and if a match is found, it will increment the
twoCounter.
 *                4) It will traverse the binary search tree that stores the states that take 3
 *                        moves to solve, and if a match is found, it will increment the
repeatCounter.
 *                5) If it doesn't find a match in any of these trees, finally it will be
 *                        entered into the tree designated for storing all the states that take 3
moves to solve.
 *                When the program is done doing this for every possible outcome while x = 3,
 *                it will move on to x = 4 and perform the same task for all possible outcomes of 4
moves.
 *                It will do this until all possible outcomes are covered, or when x = 20.
 *
 * */
package cube;

/*Import the BigInteger library*/
import java.math.BigInteger;
```

```
import java.util.ArrayList;


/*ThreeByThree class*/
public class ThreeByThree {

	/*Timers*/
	static long startTime;
	static long intermediateTime;
	static long endTime;

	/*Counters*/
	static BigInteger[] counterArray = new BigInteger[20];
	static BigInteger count = new BigInteger("1");
	static BigInteger zero = new BigInteger("0");
	static BigInteger one = new BigInteger("1");
	static BigInteger ten = new BigInteger("10");
	static BigInteger eightteen = new BigInteger("18");
	static BigInteger uniqueCounter = new BigInteger("0");
	static BigInteger repeatCounter = new BigInteger("0");

	/*Array List of storage trees*/
	static ArrayList<BinarySearchTree<BigInteger>> trees = new
ArrayList<BinarySearchTree<BigInteger>>();

	/*Reset all counters*/
	public static void resetCount(){
		count = one;
		uniqueCounter = zero;
		repeatCounter = zero;
		for(int i = 0; i < counterArray.length; i++)
			counterArray[i] = new BigInteger("0");
	}

	/*Returns a particular counter result*/
	public static BigInteger counterReturn(int i){
		return counterArray[i];
	}



	/*Increments the proper counter*/
	public static void counterIncrementer(int i){
		counterArray[i] = counterArray[i].add(one);
```

```
	}

	/*Prints the results*/
	public static void printResults(int i){
		int j = 0;
		int marker = 0;
		if(i == 1)
			System.out.println("Starting from a solved cube, when there is " + i + "
move (18^" + i + " = " + count + " total outcomes), ");
		else if(i == 0)
			System.out.println("Starting from a solved cube, when there are " + i + "
moves (18^" + i + " = " + count + " total outcome), ");
		else
			System.out.println("Starting from a solved cube, when there are " + i + "
moves (18^" + i + " = " + count + " total outcomes), ");
		if(i == 0)
			System.out.println("there was 1 unique case that took 0 moves to
solve.");
		while(marker < i){
			System.out.print("there were ");
			System.out.print(counterReturn(j));
			if(j == 1)
				System.out.println(" cases that took " + j + " move to solve, ");
			else
				System.out.println(" cases that took " + j + " moves to solve, ");
			j++;
			marker++;
		}
		if(i != 0){
			if(i == 1){
				System.out.println(uniqueCounter.toString() + " were unique
cases that took " + i + " move to solve,");
				System.out.println("while " + repeatCounter.toString() + " of them
were repeat cases that also took " + i + " move to solve.");
			}
			else{
				System.out.println(uniqueCounter.toString() + " were unique
cases that took " + i + " moves to solve,");
				System.out.println("while " + repeatCounter.toString() + " of them
were repeat cases that also took " + i + " moves to solve.");
			}
		}
		endTime = System.currentTimeMillis();
```

```
		System.out.println("Execution time for this module: " + (endTime -
intermediateTime) );
		System.out.println("Total execution time so far: " + (endTime - startTime));
		System.out.println("----------------------------------------------------------------");
	}

	/*Intakes the moveSequence value and performs the appropriate orientation
	  method on the cube, represented as an array*/
	public static void switcher(int moveSequence, int[] array){
		switch(moveSequence){
			case 1:  OrientationMethods.left(array);
				break;
			case 2:  OrientationMethods.leftPrime(array);
				break;
			case 3:  OrientationMethods.right(array);
				break;
			case 4:  OrientationMethods.rightPrime(array);
				break;
			case 5:  OrientationMethods.up(array);
				break;
			case 6:  OrientationMethods.upPrime(array);
				break;
			case 7:  OrientationMethods.down(array);
				break;
			case 8:  OrientationMethods.downPrime(array);
				break;
			case 9:  OrientationMethods.back(array);
				break;
			case 10: OrientationMethods.backPrime(array);
				break;
			case 11: OrientationMethods.front(array);
				break;
			case 12: OrientationMethods.frontPrime(array);
				break;
			case 13: OrientationMethods.e(array);
				break;
			case 14: OrientationMethods.ePrime(array);
				break;
			case 15: OrientationMethods.m(array);
				break;
			case 16: OrientationMethods.mPrime(array);
				break;
			case 17: OrientationMethods.s(array);
				break;
```

```
                        case 18: OrientationMethods.sPrime(array);
                                        break;
                        default: break;
                }
        }

        /*Converts the array of cube states to a unique
         * BigInteger key*/
        public static BigInteger convert(int[] array){
                BigInteger exponent = new BigInteger("1");
                BigInteger total = new BigInteger("0");
                for(int i = 0; i < array.length; i++){
                        total = total.add((BigInteger.valueOf(array[array.length - i -
1]).multiply(exponent)));
                        exponent = exponent.multiply(ten);
                }
                return total;
        }

        /*Takes in the current cube as a BigInteger, int input, int searchUpTo, and the tree to be
searched.
         * Searches for a match in the inputted tree. If a match is found, it'll return true,
         * otherwise false*/
        public static boolean search(BigInteger cube, int input, int searchUpTo,
BinarySearchTree<BigInteger> tree){
                if(i > searchUpTo && tree.find(cube)){
                        counterIncrementer(searchUpTo);
                        return true;
                }
                return false;
        }

/*Takes in the current cube as a BigInteger, and int input. It returns true if any search
 * method call returns true, otherwise it returns false*/
        public static boolean test(BigInteger cube, int input){
                for(int a = 0; a < 20; a++){
                        if(search(cube, input, a, trees.get(a))){
                                return true;
                        }
                }
                return false;
        }

        /*Goes through every possible outcome for 'input' amount of moves, and
```

```
 * for each outcome, either finds a match and increments the appropriate
 * count, or inserts the outcome into 'tree'. After exhausting all outcomes,
 * it prints the results and resets all counts.*/
public static void execute(int input, BinarySearchTree<BigInteger> tree){
	/*Set the intermediateTime so each module can be timed*/
	intermediateTime = System.currentTimeMillis();
	/*Based on input, multiply count by 18 the current amount of
	 * times to establish the total amount of times to iterate*/
	for(int i = 0; i < input; i++)
		count = count.multiply(eightteen);
	/*For each possible outcome when doing 18^(input) moves..*/
	for(BigInteger i = BigInteger.ZERO; i.compareTo(count) < 0; i = i.add(one)){

		/*Declare a 'found' boolean as false (nothings been found yet)*/
		boolean found = false;
		/*Create a string that represents the current i value in base 18*/
		String str = i.toString(18);
		/*Add 0's to the front of that string until it is the right length (if
		 * necessary)*/
		while(str.length() < input)
			str = "0" + str;
		/*Create a solved cube*/
		int[] Cube = {1,1,1,1,1,1,1,1,1,
				2,2,2,3,3,3,4,4,4,
				2,2,2,3,3,3,4,4,4,
				2,2,2,3,3,3,4,4,4,
				5,5,5,5,5,5,5,5,5,
				6,6,6,6,6,6,6,6,6};
		/*Perform the appropriate moves on the solved cube*/
		for(int j = 0; j < input; j++)
			switcher((Character.getNumericValue(str.charAt(j)) + 1), Cube);
		/*Reorient the cube*/
		OrientationMethods.reorient(Cube);
		/*Convert the cube to a BigInteger*/
		BigInteger cube = convert(Cube);
		/*Search for a match of the cube in each of the trees that
		 * have already been created, and if a match is found,
		 * increment the appropriate counter and set found to true*/
		found = test(cube, input);

		/*If no match is found in the already created trees, search
		 * for a match in the currently-being-created tree. If a
		 * match is found, increment the repeatCounter and set
		 * found to true*/
```

```
                if(found == false && tree.find(cube)){
                        repeatCounter = repeatCounter.add(one);
                        found = true;
                }
                /*If no match is found, insert the cube into the
                 * currently-being-created tree, and increment the
                 * uniqueCounter*/
                if(found == false){
                        tree.insert(cube);
                        uniqueCounter = uniqueCounter.add(one);
                }
                /*Reset found to false*/
                found = false;
        }
        /*Print the results and reset the counts*/
        printResults(input);
        resetCount();
    }

    /*puts all the components of the program into a simple method*/
    public static void run(){
        for(int i = 0; i < counterArray.length; i++)
                counterArray[i] = new BigInteger("0");
        for(int i = 0; i < 21; i++)
                trees.add(new BinarySearchTree<BigInteger>());
        startTime = System.currentTimeMillis();
        for(int i = 0; i < 21; i++)
                execute(i, trees.get(i));
    }

    /*Runs the program*/
    public static void main(String[] args){
        run();
    }
}
```

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: ThreeByThreeCube
 * Class: OrientationMethods
 * Purpose of this Program: The purpose of this program is to systematically
 * go through every possible case a 3x3x3 Rubik's cube can exist in,
 * in order to be able to say the following:
 *                "when there are x moves performed on a 3x3x3 Rubik's Cube,
 *                there are ____ cases that take y moves to solve."
 *  A chart that will display the results is set up with incrementing x values
 *  in each column, incrementing y values for each row, and the blank will be
 *  filled in with the correct value to appropriately complete the statement.
 *  Purpose of this Class: This class exists to provide the main method with
 *  every possible movement the 3x3x3 Rubik's Cube can make, as well as x, y,
 *  z entire-cube rotations, and lastly a re-orient method.
 *
 * */
package cube;

public class OrientationMethods {

	/*performs a left slice clock-wise quarter turn.*/
	public static void left(int[] array){
		int temp = array[0];
		array[0] = array[45];
		array[45] = array[36];
		array[36] = array[12];
		array[12] = temp;

		temp = array[3];
		array[3] = array[48];
		array[48] = array[39];
		array[39] = array[21];
		array[21] = temp;

		temp = array[6];
		array[6] = array[51];
		array[51] = array[42];
		array[42] = array[30];
		array[30] = temp;

		temp = array[9];
		array[9] = array[27];
		array[27] = array[29];
```

```
        array[29] = array[11];
        array[11] = temp;

        temp = array[10];
        array[10] = array[18];
        array[18] = array[28];
        array[28] = array[20];
        array[20] = temp;
}

/*performs a left slice counter-clock-wise quarter turn.*/
public static void leftPrime(int[] array){
        int temp = array[0];
        array[0] = array[12];
        array[12] = array[36];
        array[36] = array[45];
        array[45] = temp;

        temp = array[3];
        array[3] = array[21];
        array[21] = array[39];
        array[39] = array[48];
        array[48] = temp;

        temp = array[6];
        array[6] = array[30];
        array[30] = array[42];
        array[42] = array[51];
        array[51] = temp;

        temp = array[9];
        array[9] = array[11];
        array[11] = array[29];
        array[29] = array[27];
        array[27] = temp;

        temp = array[10];
        array[10] = array[20];
        array[20] = array[28];
        array[28] = array[18];
        array[18] = temp;
}

/*performs a right slice clock-wise quarter turn.*/
```

```
public static void right(int[] array){
	int temp = array[53];
	array[53] = array[8];
	array[8] = array[32];
	array[32] = array[44];
	array[44] = temp;

	temp = array[50];
	array[50] = array[5];
	array[5] = array[23];
	array[23] = array[41];
	array[41] = temp;

	temp = array[47];
	array[47] = array[2];
	array[2] = array[14];
	array[14] = array[38];
	array[38] = temp;

	temp = array[33];
	array[33] = array[35];
	array[35] = array[17];
	array[17] = array[15];
	array[15] = temp;

	temp = array[24];
	array[24] = array[34];
	array[34] = array[26];
	array[26] = array[16];
	array[16] = temp;
}

/*performs a right slice counter-clock-wise quarter turn.*/
public static void rightPrime(int[] array){
	int temp = array[53];
	array[53] = array[44];
	array[44] = array[32];
	array[32] = array[8];
	array[8] = temp;

	temp = array[50];
	array[50] = array[41];
	array[41] = array[23];
	array[23] = array[5];
```

```
        array[5] = temp;

        temp = array[47];
        array[47] = array[38];
        array[38] = array[14];
        array[14] = array[2];
        array[2] = temp;

        temp = array[33];
        array[33] = array[15];
        array[15] = array[17];
        array[17] = array[35];
        array[35] = temp;

        temp = array[24];
        array[24] = array[16];
        array[16] = array[26];
        array[26] = array[34];
        array[34] = temp;
}

/*performs an up slice clock-wise quarter turn.*/
public static void up(int[] array){
        int temp = array[6];
        array[6] = array[29];
        array[29] = array[38];
        array[38] = array[15];
        array[15] = temp;

        temp = array[7];
        array[7] = array[20];
        array[20] = array[37];
        array[37] = array[24];
        array[24] = temp;

        temp = array[8];
        array[8] = array[11];
        array[11] = array[36];
        array[36] = array[33];
        array[33] = temp;

        temp = array[12];
        array[12] = array[30];
        array[30] = array[32];
```

```
        array[32] = array[14];
        array[14] = temp;

        temp = array[13];
        array[13] = array[21];
        array[21] = array[31];
        array[31] = array[23];
        array[23] = temp;
}

/*performs an up slice counter-clock-wise quarter turn.*/
public static void upPrime(int[] array){
        int temp = array[6];
        array[6] = array[15];
        array[15] = array[38];
        array[38] = array[29];
        array[29] = temp;

        temp = array[7];
        array[7] = array[24];
        array[24] = array[37];
        array[37] = array[20];
        array[20] = temp;

        temp = array[8];
        array[8] = array[33];
        array[33] = array[36];
        array[36] = array[11];
        array[11] = temp;

        temp = array[12];
        array[12] = array[14];
        array[14] = array[32];
        array[32] = array[30];
        array[30] = temp;

        temp = array[13];
        array[13] = array[23];
        array[23] = array[31];
        array[31] = array[21];
        array[21] = temp;
}

/*performs a down slice clock-wise quarter turn.*/
```

```
public static void down(int[] array){
	int temp = array[0];
	array[0] = array[17];
	array[17] = array[44];
	array[44] = array[27];
	array[27] = temp;

	temp = array[1];
	array[1] = array[26];
	array[26] = array[43];
	array[43] = array[18];
	array[18] = temp;

	temp = array[2];
	array[2] = array[35];
	array[35] = array[42];
	array[42] = array[9];
	array[9] = temp;

	temp = array[45];
	array[45] = array[51];
	array[51] = array[53];
	array[53] = array[47];
	array[47] = temp;

	temp = array[46];
	array[46] = array[48];
	array[48] = array[52];
	array[52] = array[50];
	array[50] = temp;
}

/*performs a down slice counter-clock-wise quarter turn.*/
public static void downPrime(int[] array){
	int temp = array[0];
	array[0] = array[27];
	array[27] = array[44];
	array[44] = array[17];
	array[17] = temp;

	temp = array[1];
	array[1] = array[18];
	array[18] = array[43];
	array[43] = array[26];
```

```
        array[26] = temp;

        temp = array[2];
        array[2] = array[9];
        array[9] = array[42];
        array[42] = array[35];
        array[35] = temp;

        temp = array[45];
        array[45] = array[47];
        array[47] = array[53];
        array[53] = array[51];
        array[51] = temp;

        temp = array[46];
        array[46] = array[50];
        array[50] = array[52];
        array[52] = array[48];
        array[48] = temp;
}

/*performs a front slice clock-wise quarter turn.*/
public static void front(int[] array){
        int temp = array[27];
        array[27] = array[47];
        array[47] = array[33];
        array[33] = array[30];
        array[30] = temp;

        temp = array[28];
        array[28] = array[46];
        array[46] = array[34];
        array[34] = array[31];
        array[31] = temp;

        temp = array[29];
        array[29] = array[45];
        array[45] = array[35];
        array[35] = array[32];
        array[32] = temp;

        temp = array[36];
        array[36] = array[42];
        array[42] = array[44];
```

```
        array[44] = array[38];
        array[38] = temp;

        temp = array[37];
        array[37] = array[39];
        array[39] = array[43];
        array[43] = array[41];
        array[41] = temp;
}

/*performs a front slice counter-clock-wise quarter turn.*/
public static void frontPrime(int[] array){
        int temp = array[27];
        array[27] = array[30];
        array[30] = array[33];
        array[33] = array[47];
        array[47] = temp;

        temp = array[28];
        array[28] = array[31];
        array[31] = array[34];
        array[34] = array[46];
        array[46] = temp;

        temp = array[29];
        array[29] = array[32];
        array[32] = array[35];
        array[35] = array[45];
        array[45] = temp;

        temp = array[36];
        array[36] = array[38];
        array[38] = array[44];
        array[44] = array[42];
        array[42] = temp;

        temp = array[37];
        array[37] = array[41];
        array[41] = array[43];
        array[43] = array[39];
        array[39] = temp;
}

/*performs a back slice clock-wise quarter turn.*/
```

```
public static void back(int[] array){
	int temp = array[9];
	array[9] = array[12];
	array[12] = array[15];
	array[15] = array[53];
	array[53] = temp;

	temp = array[10];
	array[10] = array[13];
	array[13] = array[16];
	array[16] = array[52];
	array[52] = temp;

	temp = array[11];
	array[11] = array[14];
	array[14] = array[17];
	array[17] = array[51];
	array[51] = temp;

	temp = array[0];
	array[0] = array[6];
	array[6] = array[8];
	array[8] = array[2];
	array[2] = temp;

	temp = array[1];
	array[1] = array[3];
	array[3] = array[7];
	array[7] = array[5];
	array[5] = temp;
}

/*performs a back slice counter-clock-wise quarter turn.*/
public static void backPrime(int[] array){
	int temp = array[9];
	array[9] = array[53];
	array[53] = array[15];
	array[15] = array[12];
	array[12] = temp;

	temp = array[10];
	array[10] = array[52];
	array[52] = array[16];
	array[16] = array[13];
```

```
        array[13] = temp;

        temp = array[11];
        array[11] = array[51];
        array[51] = array[17];
        array[17] = array[14];
        array[14] = temp;

        temp = array[0];
        array[0] = array[2];
        array[2] = array[8];
        array[8] = array[6];
        array[6] = temp;

        temp = array[1];
        array[1] = array[5];
        array[5] = array[7];
        array[7] = array[3];
        array[3] = temp;
}

/*performs a middle layer (between L and R) clock-wise(as L) quarter turn.*/
public static void m(int[] array){
        int temp = array[1];
        array[1] = array[46];
        array[46] = array[37];
        array[37] = array[13];
        array[13] = temp;

        temp = array[4];
        array[4] = array[49];
        array[49] = array[40];
        array[40] = array[22];
        array[22] = temp;

        temp = array[7];
        array[7] = array[52];
        array[52] = array[43];
        array[43] = array[31];
        array[31] = temp;
}

/*performs a middle layer(between L and R) counter-clock-wise(as L) quarter turn.*/
public static void mPrime(int[] array){
```

```
        int temp = array[1];
        array[1] = array[13];
        array[13] = array[37];
        array[37] = array[46];
        array[46] = temp;

        temp = array[4];
        array[4] = array[22];
        array[22] = array[40];
        array[40] = array[49];
        array[49] = temp;

        temp = array[7];
        array[7] = array[31];
        array[31] = array[43];
        array[43] = array[52];
        array[52] = temp;
}

/*performs a middle layer(between F and B) clock-wise(as F) quarter turn.*/
public static void s(int[] array){
        int temp = array[18];
        array[18] = array[50];
        array[50] = array[24];
        array[24] = array[21];
        array[21] = temp;

        temp = array[19];
        array[19] = array[49];
        array[49] = array[25];
        array[25] = array[22];
        array[22] = temp;

        temp = array[20];
        array[20] = array[48];
        array[48] = array[26];
        array[26] = array[23];
        array[23] = temp;
}

/*performs a middle layer(between F and B) counter-clock-wise(as F) quarter turn.*/
public static void sPrime(int[] array){
        int temp = array[18];
        array[18] = array[21];
```

```
        array[21] = array[24];
        array[24] = array[50];
        array[50] = temp;

        temp = array[19];
        array[19] = array[22];
        array[22] = array[25];
        array[25] = array[49];
        array[49] = temp;

        temp = array[20];
        array[20] = array[23];
        array[23] = array[26];
        array[26] = array[48];
        array[48] = temp;
}

/*performs a middle layer(between U and D) clock-wise(as D) quarter turn.*/
public static void e(int[] array){
        int temp = array[10];
        array[10] = array[5];
        array[5] = array[34];
        array[34] = array[39];
        array[39] = temp;

        temp = array[19];
        array[19] = array[4];
        array[4] = array[25];
        array[25] = array[40];
        array[40] = temp;

        temp = array[28];
        array[28] = array[3];
        array[3] = array[16];
        array[16] = array[41];
        array[41] = temp;
}

/*performs a middle layer(between U and D) counter-clock-wise(as D) quarter turn.*/
public static void ePrime(int[] array){
        int temp = array[10];
        array[10] = array[39];
        array[39] = array[34];
        array[34] = array[5];
```

```
        array[5] = temp;

        temp = array[19];
        array[19] = array[40];
        array[40] = array[25];
        array[25] = array[4];
        array[4] = temp;

        temp = array[28];
        array[28] = array[41];
        array[41] = array[16];
        array[16] = array[3];
        array[3] = temp;
}

/*rotate the entire cube clockwise on R.*/
public static void x(int[] array){
        leftPrime(array);
        mPrime(array);
        right(array);
}

/*rotate the entire cube counter-clockwise on R.*/
public static void xPrime(int[] array){
        left(array);
        m(array);
        rightPrime(array);
}

/*rotate the entire cube clockwise on U.*/
public static void y(int[] array){
        up(array);
        ePrime(array);
        downPrime(array);
}

/*rotate the entire cube counter-clockwise on U.*/
public static void yPrime(int[] array){
        upPrime(array);
        e(array);
        down(array);
}

/*rotate the entire cube clockwise on F.*/
```

```
	public static void z(int[] array){
		front(array);
		s(array);
		backPrime(array);
	}

	/*rotate the entire cube counter-clockwise on F.*/
	public static void zPrime(int[] array){
		frontPrime(array);
		sPrime(array);
		back(array);
	}

	/*orient the cube to standard form.*/
	public static void reorient(int[] array){
		while(array[4] != 1 || array[19] != 2){
			while(array[4] != 1){
				if(array[19] != 1 && array[25] != 1){
					x(array);
				}
				else{
					y(array);
				}
			}
			while(array[19] != 2){
				z(array);
			}
		}
	}
}
```

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: ThreeByThreeCube
 * Class: BinarySearchTree
 * Purpose of this Program: The purpose of this program is to systematically
 * go through every possible case a 3x3x3 Rubik's cube can exist in,
 * in order to be able to say the following:
 *                "when there are x moves performed on a 3x3x3 Rubik's Cube,
 *                there are ____ cases that take y moves to solve."
 *  A chart that will display the results is set up with incrementing x values
 *  in each column, incrementing y values for each row, and the blank will be
 *  filled in with the correct value to appropriately complete the statement.
 *  Purpose of this Class: This classes purpose is to implement a binary search
 *  tree, so the cube states can be stored in a tree, and the search for matches
 *  increases in run time from O(n) to O(log(n)), where n is the number of entries
 *  being stored at a given distance.
 *  */

package cube;

import java.util.LinkedList;
import java.util.Queue;

public class BinarySearchTree<T extends Comparable<T>> {

        class BinaryNode<E extends Comparable<E>>{
                protected E data;
                protected BinaryNode<E> right, left;
                protected int height;

                public BinaryNode(){
                        data = null;
                        left = null;
                        right = null;
                        height = 0;
                }

                public BinaryNode(E d){
                        data = d;
                        right = null;
                        left = null;
                }

                public BinaryNode(BinaryNode<E> p, E d, BinaryNode<E> n){
```

```
                data = d;
                right = n;
                left = p;
        }

        public BinaryNode(BinaryNode<E> p, E d, BinaryNode<E> n, int h){
                data = d;
                right = n;
                left = p;
                height = h;
        }
}

protected BinaryNode<T> root;
private int size;

public void insert(T d){
        root = insert(d, root);
        size++;
}

protected BinaryNode<T> rotateWithLeftChild(BinaryNode<T> k2){
        BinaryNode<T> k1 = k2.left;
        k2.left = k1.right;
        k1.right = k2;
        k2.height = max(height(k2.left), height(k2.right)) + 1;
        k1.height = max(height(k1.left), k2.height) + 1;
        return k1;
}

protected BinaryNode<T> rotateWithRightChild(BinaryNode<T> k1){
        BinaryNode<T> k2 = k1.right;
        k1.right = k2.left;
        k2.left = k1;
        k1.height = max(height(k1.left), height(k1.right)) + 1;
        k2.height = max(height(k2.right), k1.height) + 1;
        return k2;
}

protected BinaryNode<T> doubleWithLeftChild(BinaryNode<T> k3){
        k3.left = rotateWithRightChild(k3.left);
        return rotateWithLeftChild(k3);
}
```

```
protected BinaryNode<T> doubleWithRightChild(BinaryNode<T> k1){
	k1.right = rotateWithLeftChild(k1.right);
	return rotateWithRightChild(k1);
}

protected BinaryNode<T> insert(T d, BinaryNode<T> root){
	if(root == null)
		root = new BinaryNode<T>(d);
	else if(root.data.compareTo(d) > 0){
		root.left = insert(d, root.left);
		if(height(root.left) - height(root.right) == 2){
			if(root.left.data.compareTo(d) > 0)
				root = rotateWithLeftChild(root);
			else
				root = doubleWithLeftChild(root);
		}
	}
	else if(root.data.compareTo(d) < 0){
		root.right = insert(d, root.right);
		if(height(root.right) - height(root.left) == 2){
			if(root.right.data.compareTo(d) < 0)
				root = rotateWithRightChild(root);
			else
				root = doubleWithRightChild(root);
		}
	}
	else
		;//do nothing
	root.height = max(height(root.left), height(root.right)) + 1;
	return root;
}

protected int height(BinaryNode<T> root){
	if(root == null){
		return 0;
	}
	return root.height;
}

protected int max(int a, int b){
	return (a > b) ? a : b;
}

public boolean find(T id){
```

```
        BinarySearchTree<T>.BinaryNode<T> current = root;
        while(current != null){
                if(current.data.compareTo(id) == 0){
                        return true;
                }else if(current.data.compareTo(id) == 1){
                        current = current.left;
                }else{
                        current = current.right;
                }
        }
        return false;
}

public String preorder(){
        String str = "\n[Pre-order - size = " + size + "]\n";
        str += preorder(root);
        return str;
}

public String inorder(){
        String str = "\n[In-order - size = " + size + "]\n";
        str += inorder(root);
        return str;
}

public String postorder(){
        String str = "\n[Post-order - size = " + size + "]\n";
        str += postorder(root);
        return str;
}

protected String preorder(BinaryNode <T> root){
        String msg = "";
        if(root != null){
                msg += root.data + " ";
                msg += preorder(root.left);
                msg += preorder(root.right);
        }
        return msg;

}

protected String inorder(BinaryNode <T> root){
        String msg = "";
```

```
        if(root != null){
                msg += inorder(root.left);
                msg += root.data + " ";
                msg += inorder(root.right);
        }
        return msg;
}

protected String postorder(BinaryNode <T> root){
        String msg = "";
        if(root != null){
                msg += postorder(root.left);
                msg += postorder(root.right);
                msg += root.data + " ";
        }
        return msg;
}

protected void levelOrderPrint(BinaryNode <T> root){
        Queue<BinaryNode<T>> queue = new LinkedList<BinaryNode<T>>();
        LinkedList<T> list = new LinkedList<T>();
        queue.add(root);
        while(!queue.isEmpty()){
                BinaryNode<T> item = (BinaryNode<T>) queue.remove();
                if(item != null){
                        list.addLast(item.data);
                        if(item.left != null)
                                queue.add(item.left);
                        if(item.right != null)
                                queue.add(item.right);
                }
        }
System.out.println(list);
}

protected BinaryNode<T> insertLevel(T d, BinaryNode<T> root){
        if(root == null)
                root = new BinaryNode<T>(d);
        else if(root.left == null){
                root.left = new BinaryNode<T>(d);
                insert(d, root.left);
        }
        else if(root.right == null){
                root.right = new BinaryNode<T>(d);
```

```
                insert(d, root.right);
            }
            return root;
        }

        public void insertLevel(T d){
            insertLevel(d, root);
            size++;
        }

        public void levelOrderPrint(){
            levelOrderPrint(root);
        }

        public void levelOrderInsert(T d) {
    size++;
    if(root == null)
        root = new BinaryNode<T>(d);
    else {
        Queue<BinaryNode<T>> queue = new LinkedList<BinaryNode<T>>();

        queue.add(root);
        while(!queue.isEmpty()) {
            BinaryNode<T> item = queue.remove();
            if(item != null) {
                if(item.left != null){
                    queue.add(item.left);
                    if(item.right != null)
                        queue.add(item.right);
                    else{
                        item.right = new BinaryNode<T>(d);
                        return;
                    }
                }
                else{
                    item.left = new BinaryNode<T>(d);
                    return;
                }
            }
        }
    }
  }
}
```

Sample Output

```
Starting from a solved cube, when there are 0 moves (18^0 = 1 total outcome),
there was 1 unique case that took 0 moves to solve.
Execution time for this module: 2
Total execution time so far: 2
----------------------------------------------------------------
Starting from a solved cube, when there is 1 move (18^1 = 18 total outcomes),
there were 0 cases that took 0 moves to solve,
18 were unique cases that took 1 move to solve,
while 0 of them were repeat cases that also took 1 move to solve.
Execution time for this module: 4
Total execution time so far: 6
----------------------------------------------------------------
Starting from a solved cube, when there are 2 moves (18^2 = 324 total outcomes),
there were 18 cases that took 0 moves to solve,
there were 36 cases that took 1 move to solve,
243 were unique cases that took 2 moves to solve,
while 27 of them were repeat cases that also took 2 moves to solve.
Execution time for this module: 24
Total execution time so far: 30
----------------------------------------------------------------
Starting from a solved cube, when there are 3 moves (18^3 = 5832 total outcomes),
there were 36 cases that took 0 moves to solve,
there were 720 cases that took 1 move to solve,
there were 1188 cases that took 2 moves to solve,
3240 were unique cases that took 3 moves to solve,
while 648 of them were repeat cases that also took 3 moves to solve.
Execution time for this module: 161
Total execution time so far: 191
----------------------------------------------------------------
Starting from a solved cube, when there are 4 moves (18^4 = 104976 total outcomes),
there were 720 cases that took 0 moves to solve,
there were 3600 cases that took 1 move to solve,
there were 18360 cases that took 2 moves to solve,
there were 25920 cases that took 3 moves to solve,
42815 were unique cases that took 4 moves to solve,
while 13561 of them were repeat cases that also took 4 moves to solve.
Execution time for this module: 1384
Total execution time so far: 1575
----------------------------------------------------------------
Starting from a solved cube, when there are 5 moves (18^5 = 1889568 total outcomes),
there were 3600 cases that took 0 moves to solve,
```

there were 42624 cases that took 1 move to solve,
there were 130032 cases that took 2 moves to solve,
there were 392592 cases that took 3 moves to solve,
there were 510624 cases that took 4 moves to solve,
558930 were unique cases that took 5 moves to solve,
while 251166 of them were repeat cases that also took 5 moves to solve.
Execution time for this module: 13528
Total execution time so far: 15103

Starting from a solved cube, when there are 6 moves (18^6 = 34012224 total outcomes),
there were 42624 cases that took 0 moves to solve,
there were 312480 cases that took 1 move to solve,
there were 1428168 cases that took 2 moves to solve,
there were 3403824 cases that took 3 moves to solve,
there were 7935120 cases that took 4 moves to solve,
there were 9332880 cases that took 5 moves to solve,
7234281 were unique cases that took 6 moves to solve,
while 4322847 of them were repeat cases that also took 6 moves to solve.
Execution time for this module: 259436
Total execution time so far: 274539

Appendix B

TwoByTwoCube

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: TwoByTwoCube
 * Class: TwoByTwo
 * Purpose of this Program: The purpose of this program is to systematically
 *                go through every possible case a 2x2x2 Rubik's cube can exist in,
 *                in order to be able to say the following:
 *                When there are x moves performed on a 2x2x2 Rubik's Cube,
 *                there are ____ cases that take y moves to solve."
 *        A chart that will display the results is set up with incrementing x values
 *        in each column, incrementing y values for each row, and the blank will be
 *        filled in with the correct value to appropriately complete the statement.
 * Purpose of this Class: This class runs the main program which works like this:
 *        Starting with x = 0, all the way up to x = 14, the program will perform all
 *        possible sequences of x moves, then try to find matches amongst all states that
 *        have already been stored, and if so, will count it accordingly.
 *        For example, When x = 3, the program will compute every outcome that results
 *        from performing 3 moves on the 2x2x2 Rubik's Cube. Each outcome will go through
 *        the following:
 *        1) It will traverse the binary search tree that stores the 1 state that
 *                takes 0 moves to solve, and if a match is found, it will increment the
 *                zeroCounter.
 *        2) It will traverse the binary search tree that stores all 6 states that
 *                take 1 move to solve, and if a match is found, it will increment the oneCounter.
 *        3) It will traverse the binary search tree that stores all 27 states that take 2
 *                moves to solve, and if a match is found, it will increment the twoCounter.
 *        4) It will traverse the binary search tree that stores the states that take 3
 *                moves to solve, and if a match is found, it will increment the repeatCounter.
 *        5) If it doesn't find a match in any of these trees, finally it will be
 *                entered into the tree designated for storing all the states that take 3 moves to
solve,
 *                and it will increment the uniqueCounter.
 *                When the program is done doing this for every possible outcome while x = 3, it
will
 *                print the results to that point, reset the counts, and then it will move on to x = 4
 *                and perform the same task for all possible outcomes of 4 moves. It will do this
until
 *                all possible outcomes are covered, or when x = 14.
 *
 * */
package cube;

/*Import the BigInteger library*/
import java.math.BigInteger;
```

```
import java.util.ArrayList;

/*TwoByTwo class*/
public class TwoByTwo {

	/*Timers*/
	static long startTime;
	static long intermediateTime;
	static long endTime;

	/*Counters*/
	static BigInteger[] counterArray = new BigInteger[14];
	static BigInteger count = new BigInteger("1");
	static BigInteger zero = new BigInteger("0");
	static BigInteger one = new BigInteger("1");
	static BigInteger ten = new BigInteger("10");
	static BigInteger twelve = new BigInteger("12");
	static BigInteger uniqueCounter = new BigInteger("0");
	static BigInteger repeatCounter = new BigInteger("0");

	/*Array List of storage trees*/
	static ArrayList<BinarySearchTree<BigInteger>> trees = new
ArrayList<BinarySearchTree<BigInteger>>();

	/*Reset all counters*/
	public static void resetCount(){
		count = one;
		uniqueCounter = zero;
		repeatCounter = zero;
		for(int i = 0; i < counterArray.length; i++)
			counterArray[i] = new BigInteger("0");
	}

	/*Returns a particular counter result*/
	public static BigInteger counterReturn(int i){
		return counterArray[i];
	}
	/*Increments the proper counter*/
	public static void counterIncrementer(int i){
		counterArray[i] = counterArray[i].add(one);
	}

	/*Prints the results*/
	public static void printResults(int i){
```

```
		int j = 0;
		int marker = 0;
		if(i == 1){
			System.out.println("Starting from a solved cube, when there is " + i + "
move (12^" + i + " = " + count + " total outcomes), ");
		}
		else if(i == 0)
			System.out.println("Starting from a solved cube, when there are " + i + "
moves (12^" + i + " = " + count + " total outcome), ");
		else{
			System.out.println("Starting from a solved cube, when there are " + i + "
moves (12^" + i + " = " + count + " total outcomes), ");
		}
		if(i == 0){
			System.out.println("there was 1 unique case that took 0 moves to
solve.");
		}
		while(marker < i){
			System.out.print("there were ");
			System.out.print(counterReturn(j));
			if(j == 1)
				System.out.println(" cases that took " +  j + " move to solve, ");
			else
				System.out.println(" cases that took " +  j + " moves to solve, ");
			j++;
			marker++;
		}
		if(i != 0){
			if(i == 1){
				System.out.println(uniqueCounter.toString() + " were unique
cases that took " + i + " move to solve,");
				System.out.println("while " + repeatCounter.toString() + " of them
were repeat cases that also took " + i + " move to solve.");
			}
			else{
				System.out.println(uniqueCounter.toString() + " were unique
cases that took " + i + " moves to solve,");
				System.out.println("while " + repeatCounter.toString() + " of them
were repeat cases that also took " + i + " moves to solve.");
			}
		}
		endTime = System.currentTimeMillis();
		System.out.println("Execution time for this module: " + (endTime -
intermediateTime));
```

```
		System.out.println("Total execution time so far: " + (endTime - startTime) );
		System.out.println("----------------------------------------------------------------");
	}

	/*Intakes the moveSequence value and performs the appropriate orientation
	  method on the cube, represented as an array*/
	public static void switcher(int moveSequence, int[] array){
		switch(moveSequence){
			case 1: OrientationMethods.left(array);
				break;
			case 2: OrientationMethods.leftPrime(array);
				break;
			case 3: OrientationMethods.right(array);
				break;
			case 4: OrientationMethods.rightPrime(array);
				break;
			case 5: OrientationMethods.up(array);
				break;
			case 6: OrientationMethods.upPrime(array);
				break;
			case 7: OrientationMethods.down(array);
				break;
			case 8: OrientationMethods.downPrime(array);
				break;
			case 9: OrientationMethods.front(array);
				break;
			case 10:OrientationMethods.frontPrime(array);
				break;
			case 11:OrientationMethods.back(array);
				break;
			case 12:OrientationMethods.backPrime(array);
				break;
			default:break;
		}
	}

	/*Converts the array of cube states to a unique BigInteger key*/
	public static BigInteger convert(int[] array){
		BigInteger exponent = new BigInteger("1");
		BigInteger total = new BigInteger("0");
		for(int i = 0; i < array.length; i++){
			total = total.add((BigInteger.valueOf(array[array.length - i -
1]).multiply(exponent)));
			exponent = exponent.multiply(ten);
```

```
		}
		return total;
	}

	/*Takes in the current cube as a BigInteger, int input, int searchUpTo, and the tree to be
searched.
	 * Searches for a match in the inputted tree. If a match is found, it'll return true,
otherwise false*/
	public static boolean search(BigInteger cube, int input, int searchUpTo,
BinarySearchTree<BigInteger> tree){
		if(input > searchUpTo && tree.find(cube)){
			counterIncrementer(searchUpTo);
			return true;
		}
		return false;
	}
	/*Takes in the current cube as a BigInteger, and int input. It returns true if any search
	 * method call returns true, otherwise it returns false*/
	public static boolean test(BigInteger cube, int input){
		for(int a = 0; a < 14; a++){
			if(search(cube, input, a, trees.get(a)))
				return true;
		}
		return false;
	}

	/*Goes through every possible outcome for 'input' amount of moves, and
	 * for each outcome, either finds a match and increments the appropriate
	 * count, or inserts the outcome into 'tree'. After exhausting all outcomes,
	 * it prints the results and resets all counts.*/
	public static void execute(int input, BinarySearchTree<BigInteger> tree){
		/*Set the intermediateTime so each module can be timed*/
		intermediateTime = System.currentTimeMillis();
		/*Based on input, multiply count by 12 the correct amount of
		 * times to establish the total amount of times to iterate*/
		for(int i = 0; i < input; i++)
			count = count.multiply(twelve);
		/*For each possible outcome when doing 12^(input) moves.. */
		for(BigInteger i = BigInteger.ZERO; i.compareTo(count) < 0; i = i.add(one)){
			/*Declare a 'found' boolean as false (nothings been found yet)*/
			boolean found = false;
			/*Create a string that represents the current i value in base 12*/
			String str = i.toString(12);
```

```
                /*Add 0's to the front of that string until it is the right length (if
necessary)*/
                while(str.length() < input)
                        str = "0" + str;
                /*Create a solved cube*/
                int[] Cube = {1,1,1,1,2,2,3,3,4,4,2,2,3,3,4,4,5,5,5,5,6,6,6,6};
                /*Perform the appropriate moves on the solved cube*/
                for(int j = 0; j < input; j++)
                        switcher((Character.getNumericValue(str.charAt(j)) + 1), Cube);
                /*Reorient the cube*/
                OrientationMethods.reorient(Cube);
                /*Convert the cube to a BigInteger*/
                BigInteger cube = convert(Cube);
                /*Search for a match of the cube in each of the trees that
                 * have already been created, and if a match is found,
                 * increment the appropriate counter and set found to true*/
                found = test(cube, input);
                /*If no match is found in the already created trees, search
                 * for a match in the currently-being-created tree. If a
                 * match is found, increment the repeat counter and set
                 * found to true*/
                if(found == false && tree.find(cube)){
                        repeatCounter = repeatCounter.add(one);
                        found = true;
                }
                /*If no match has been found, insert the cube into the
                 * currently-being-created tree, and increment the
                 * unique counter*/
                if(found == false){
                        tree.insert(cube);
                        uniqueCounter = uniqueCounter.add(one);
                }
                /*Reset found to false*/
                found = false;
        }
        /*Print results and reset the counts*/
        printResults(input);
        resetCount();
    }

    /*Puts all the components of the program into a simple method*/
    public static void run(){
        for(int i = 0; i < counterArray.length; i++)
                counterArray[i] = new BigInteger("0");
```

```
		for(int i = 0; i < 15; i ++)
			trees.add(new BinarySearchTree<BigInteger>());
		startTime = System.currentTimeMillis();
		for(int i = 0; i < 15; i++)
			execute(i, trees.get(i));
	}

	/*Runs the program*/
	public static void main(String[] args){
		run();
	}
}
```

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: TwoByTwoCube
 * Class: OrientationMethods
 * Purpose of this Program: The purpose of this program is to systematically
 *                go through every possible case a 2x2x2 Rubik's cube can exist in,
 *                in order to be able to say the following:
 *                "when there are x moves performed on a 2x2x2 Rubik's Cube,
 *                there are ____ cases that take y moves to solve."
 *        A chart that will display the results is set up with incrementing x values
 *        in each column, incrementing y values for each row, and the blank will be
 *        filled in with the correct value to appropriately complete the statement.
 *  Purpose of this Class: This class exists to provide the main method with
 *        every possible movement the 2x2x2 Rubik's Cube can make, as well as x, y,
 *        z entire-cube rotations, and lastly a re-orient method.
 * */
package cube;

/*OrientationMethods class*/
public class OrientationMethods {

        /*Performs a left slice clock-wise quarter turn*/
        public static void left(int[] array){
                int temp = array[0];
                array[0] = array[20];
                array[20] = array[16];
                array[16] = array[6];
                array[6] = temp;

                temp = array[2];
                array[2] = array[22];
                array[22] = array[18];
                array[18] = array[12];
                array[12] = temp;

                temp = array[4];
                array[4] = array[10];
                array[10] = array[11];
                array[11] = array[5];
                array[5] = temp;
        }

        /*Performs a left slice counter-clock-wise quarter turn*/
        public static void leftPrime(int[] array){
```

```
        int temp = array[0];
        array[0] = array[6];
        array[6] = array[16];
        array[16] = array[20];
        array[20] = temp;

        temp = array[2];
        array[2] = array[12];
        array[12] = array[18];
        array[18] = array[22];
        array[22] = temp;

        temp = array[4];
        array[4] = array[5];
        array[5] = array[11];
        array[11] = array[10];
        array[10] = temp;
}

/*Performs a right slice clock-wise quarter turn*/
public static void right(int[] array){
        int temp = array[1];
        array[1] = array[7];
        array[7] = array[17];
        array[17] = array[21];
        array[21] = temp;

        temp = array[3];
        array[3] = array[13];
        array[13] = array[19];
        array[19] = array[23];
        array[23] = temp;

        temp = array[14];
        array[14] = array[15];
        array[15] = array[9];
        array[9] = array[8];
        array[8] = temp;
}

/*Performs a right slice counter-clock-wise quarter turn*/
public static void rightPrime(int[] array){
        int temp = array[1];
        array[1] = array[21];
```

```
        array[21] = array[17];
        array[17] = array[7];
        array[7] = temp;

        temp = array[3];
        array[3] = array[23];
        array[23] = array[19];
        array[19] = array[13];
        array[13] = temp;

        temp = array[14];
        array[14] = array[8];
        array[8] = array[9];
        array[9] = array[15];
        array[15] = temp;
}

/*Performs an up slice clock-wise quarter turn*/
public static void up(int[] array){
        int temp = array[6];
        array[6] = array[12];
        array[12] = array[13];
        array[13] = array[7];
        array[7] = temp;

        temp = array[5];
        array[5] = array[16];
        array[16] = array[14];
        array[14] = array[3];
        array[3] = temp;

        temp = array[11];
        array[11] = array[17];
        array[17] = array[8];
        array[8] = array[2];
        array[2] = temp;
}

/*Performs an up slice counter-clock-wise quarter turn*/
public static void upPrime(int[] array){
        int temp = array[6];
        array[6] = array[7];
        array[7] = array[13];
        array[13] = array[12];
```

```
        array[12] = temp;

        temp = array[5];
        array[5] = array[3];
        array[3] = array[14];
        array[14] = array[16];
        array[16] = temp;

        temp = array[11];
        array[11] = array[2];
        array[2] = array[8];
        array[8] = array[17];
        array[17] = temp;
}

/*Performs a down slice clock-wise quarter turn*/
public static void down(int[] array){
        int temp = array[20];
        array[20] = array[22];
        array[22] = array[23];
        array[23] = array[21];
        array[21] = temp;

        temp = array[0];
        array[0] = array[9];
        array[9] = array[19];
        array[19] = array[10];
        array[10] = temp;

        temp = array[1];
        array[1] = array[15];
        array[15] = array[18];
        array[18] = array[4];
        array[4] = temp;
}

/*Performs a down slice counter-clock-wise quarter turn*/
public static void downPrime(int[] array){
        int temp = array[20];
        array[20] = array[21];
        array[21] = array[23];
        array[23] = array[22];
        array[22] = temp;
```

```
        temp = array[0];
        array[0] = array[10];
        array[10] = array[19];
        array[19] = array[9];
        array[9] = temp;

        temp = array[1];
        array[1] = array[4];
        array[4] = array[18];
        array[18] = array[15];
        array[15] = temp;
}

/*Performs a front slice clock-wise quarter turn*/
public static void front(int[] array){
        int temp = array[16];
        array[16] = array[18];
        array[18] = array[19];
        array[19] = array[17];
        array[17] = temp;

        temp = array[10];
        array[10] = array[21];
        array[21] = array[14];
        array[14] = array[12];
        array[12] = temp;

        temp = array[11];
        array[11] = array[20];
        array[20] = array[15];
        array[15] = array[13];
        array[13] = temp;
}

/*Performs a front slice counter-clock-wise quarter turn*/
public static void frontPrime(int[] array){
        int temp = array[16];
        array[16] = array[17];
        array[17] = array[19];
        array[19] = array[18];
        array[18] = temp;

        temp = array[10];
        array[10] = array[12];
```

```
        array[12] = array[14];
        array[14] = array[21];
        array[21] = temp;

        temp = array[11];
        array[11] = array[13];
        array[13] = array[15];
        array[15] = array[20];
        array[20] = temp;
}

/*Performs a back slice clock-wise quarter turn*/
public static void back(int[] array){
        int temp = array[0];
        array[0] = array[2];
        array[2] = array[3];
        array[3] = array[1];
        array[1] = temp;

        temp = array[9];
        array[9] = array[22];
        array[22] = array[5];
        array[5] = array[7];
        array[7] = temp;

        temp = array[8];
        array[8] = array[23];
        array[23] = array[4];
        array[4] = array[6];
        array[6] = temp;
}

/*Performs a back slice counter-clock-wise quarter turn*/
public static void backPrime(int[] array){
        int temp = array[0];
        array[0] = array[1];
        array[1] = array[3];
        array[3] = array[2];
        array[2] = temp;

        temp = array[9];
        array[9] = array[7];
        array[7] = array[5];
        array[5] = array[22];
```

```
        array[22] = temp;

        temp = array[8];
        array[8] = array[6];
        array[6] = array[4];
        array[4] = array[23];
        array[23] = temp;
}

/*Rotate the entire cube clockwise on R*/
public static void x(int[] array){
        leftPrime(array);
        right(array);
}

/*Rotate the entire cube counter-clockwise on R*/
public static void xPrime(int[] array){
        left(array);
        rightPrime(array);
}

/*Rotate the entire cube clockwise on U*/
public static void y(int[] array){
        up(array);
        downPrime(array);
}

/*Rotate the entire cube counter-clockwise on U*/
public static void yPrime(int[] array){
        upPrime(array);
        down(array);
}

/*Rotate the entire cube clockwise on F*/
public static void z(int[] array){
        front(array);
        backPrime(array);
}

/*Rotate the entire cube counter-clockwise on F*/
public static void zPrime(int[] array){
        frontPrime(array);
        back(array);
}
```

```
/*Reorients the cube*/
public static void reorient(int[] array){
	int i = 0;
	boolean one = (array[0] == 5 && array[4] == 2 && array[22] == 3);
	if(one)
		i = 1;
	boolean two = (array[0] == 3 && array[4] == 5 && array[22] == 2);
	if(two)
		i = 2;
	boolean three = (array[0] == 2 && array[4] == 3 && array[22] == 5);
	if(three)
		i = 3;
	boolean four = (array[2] == 2 && array[5] == 5 && array[6] == 3);
	if(four)
		i = 4;
	boolean five = (array[2] == 3 && array[5] == 2 && array[6] == 5);
	if(five)
		i = 5;
	boolean six = (array[2] == 5 && array[5] == 3 && array[6] == 2);
	if(six)
		i = 6;
	boolean seven = (array[11] == 2 && array[12] == 3 && array[16] == 5);
	if(seven)
		i = 7;
	boolean eight = (array[11] == 3 && array[12] == 5 && array[16] == 2);
	if(eight)
		i = 8;
	boolean nine = (array[11] == 5 && array[12] == 2 && array[16] == 3);
	if(nine)
		i = 9;
	boolean ten = (array[10] == 2 && array[18] == 3 && array[20] == 5);
	if(ten)
		i = 10;
	boolean eleven = (array[10] == 5 && array[18] == 2 && array[20] == 3);
	if(eleven)
		i = 11;
	boolean twelve = (array[10] == 3 && array[18] == 5 && array[20] == 2);
	if(twelve)
		i = 12;
	boolean thirteen = (array[1] == 3 && array[9] == 2 && array[23] == 5);
	if(thirteen)
		i = 13;
	boolean fourteen = (array[1] == 5 && array[9] == 3 && array[23] == 2);
```

```
if(fourteen)
	i = 14;
boolean fifteen = (array[1] == 2 && array[9] == 5 && array[23] == 3);
if(fifteen)
	i = 15;
boolean sixteen = (array[3] == 5 && array[7] == 3 && array[8] == 2);
if(sixteen)
	i = 16;
boolean seventeen = (array[3] == 2 && array[7] == 5 && array[8] == 3);
if(seventeen)
	i = 17;
boolean eighteen = (array[3] == 3 && array[7] == 2 && array[8] == 5);
if(eighteen)
	i = 18;
boolean nineteen = (array[13] == 3 && array[14] == 5 && array[17] == 2);
if(nineteen)
	i = 19;
boolean twenty = (array[13] == 2 && array[14] == 3 && array[17] == 5);
if(twenty)
	i = 20;
boolean twentyone = (array[13] == 5 && array[14] == 2 && array[17] == 3);
if(twentyone)
	i = 21;
boolean twentytwo = (array[15] == 5 && array[19] == 3 && array[21] == 2);
if(twentytwo)
	i = 22;
boolean twentythree = (array[15] == 2 && array[19] == 5 && array[21] == 3);
if(twentythree)
	i = 23;
boolean twentyfour = (array[15] == 3 && array[19] == 2 && array[21] == 5);
if(twentyfour)
	i = 24;
switch(i){
	case 1:		x(array);
				x(array);
				break;
	case 2:		xPrime(array);
				yPrime(array);
				break;
	case 3:		yPrime(array);
				x(array);
				break;
	case 4:		yPrime(array);
				break;
```

```
case 5:         xPrime(array);
                        break;
case 6:         z(array);
                        y(array);
                        y(array);
                        break;
case 7:         break;
case 8:         z(array);
                        y(array);
                        break;
case 9:         x(array);
                        yPrime(array);
                        break;
case 10:        x(array);
                        break;
case 11:        x(array);
                        x(array);
                        yPrime(array);
                        break;
case 12:        z(array);
                        break;
case 13:        xPrime(array);
                        y(array);
                        y(array);
                        break;
case 14:        zPrime(array);
                        y(array);
                        y(array);
                        break;
case 15:        z(array);
                        z(array);
                        yPrime(array);
                        break;
case 16:        y(array);
                        y(array);
                        break;
case 17:        xPrime(array);
                        zPrime(array);
                        break;
case 18:        xPrime(array);
                        y(array);
                        break;
case 19:        y(array);
                        break;
```

```
                case 20:        zPrime(array);
                                        break;
                case 21:        x(array);
                                        y(array);
                                        y(array);
                                        break;
                case 22:        x(array);
                                        y(array);
                                        break;
                case 23:        z(array);
                                        z(array);
                                        break;
                case 24:        zPrime(array);
                                        y(array);
                                        break;
                default:        break;
            }
        }
}
```

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: TwoByTwoCube
 * Class: BinarySearchTree
 * Purpose of this Program: The purpose of this program is to systematically
 *        go through every possible case a 2x2x2 Rubik's cube can exist in,
 *        in order to be able to say the following:
 *                "when there are x moves performed on a 2x2x2 Rubik's Cube,
 *                there are ____ cases that take y moves to solve."
 *        A chart that will display the results is set up with incrementing x values
 *        in each column, incrementing y values for each row, and the blank will be
 *        filled in with the correct value to appropriately complete the statement.
 *  Purpose of this Class: This classes purpose is to implement a binary search
 *        tree, so the cube states can be stored in a tree, and the search for matches
 *        increases in run time from O(n) to O(log(n)), where n is the number of entries
 *        being stored at a given distance.
 *  */

package cube;
/*Import the LinkedList and Queue libraries*/
import java.util.LinkedList;
import java.util.Queue;

/*BinarySearchTree class extends the Comparable class*/
public class BinarySearchTree<T extends Comparable<T>> {

        /*BinaryNode class extends Comparable class*/
        class BinaryNode<E extends Comparable<E>>{
                protected E data;
                protected BinaryNode<E> right, left;
                protected int height;

                /*Constructor*/
                public BinaryNode(){
                        data = null;
                        left = null;
                        right = null;
                        height = 0;
                }

                /*Constructor*/
                public BinaryNode(E d){
                        data = d;
                        right = null;
```

```
                left = null;
        }

        /*Constructor*/
        public BinaryNode(BinaryNode<E> p, E d, BinaryNode<E> n){
                data = d;
                right = n;
                left = p;
        }

        /*Constructor*/
        public BinaryNode(BinaryNode<E> p, E d, BinaryNode<E> n, int h){
                data = d;
                right = n;
                left = p;
                height = h;
        }
}

/*Root and size*/
protected BinaryNode<T> root;
private int size;

/*Insert*/
public void insert(T d){
        root = insert(d, root);
        size++;
}

/*Rotate with left child*/
protected BinaryNode<T> rotateWithLeftChild(BinaryNode<T> k2){
        BinaryNode<T> k1 = k2.left;
        k2.left = k1.right;
        k1.right = k2;
        k2.height = max(height(k2.left), height(k2.right)) + 1;
        k1.height = max(height(k1.left), k2.height) + 1;
        return k1;
}

/*Rotate with right child*/
protected BinaryNode<T> rotateWithRightChild(BinaryNode<T> k1){
        BinaryNode<T> k2 = k1.right;
        k1.right = k2.left;
        k2.left = k1;
```

```
        k1.height = max(height(k1.left), height(k1.right)) + 1;
        k2.height = max(height(k2.right), k1.height) + 1;
        return k2;
}

/*Double rotate with left child*/
protected BinaryNode<T> doubleWithLeftChild(BinaryNode<T> k3){
        k3.left = rotateWithRightChild(k3.left);
        return rotateWithLeftChild(k3);
}

/*Double rotate with right child*/
protected BinaryNode<T> doubleWithRightChild(BinaryNode<T> k1){
        k1.right = rotateWithLeftChild(k1.right);
        return rotateWithRightChild(k1);
}

/*Insert and re-balance*/
protected BinaryNode<T> insert(T d, BinaryNode<T> root){
        if(root == null)
                root = new BinaryNode<T>(d);
        else if(root.data.compareTo(d) > 0){
                root.left = insert(d, root.left);
                if(height(root.left) - height(root.right) == 2){
                        if(root.left.data.compareTo(d) > 0)
                                root = rotateWithLeftChild(root);
                        else
                                root = doubleWithLeftChild(root);
                }
        }
        else if(root.data.compareTo(d) < 0){
                root.right = insert(d, root.right);
                if(height(root.right) - height(root.left) == 2){
                        if(root.right.data.compareTo(d) < 0)
                                root = rotateWithRightChild(root);
                        else
                                root = doubleWithRightChild(root);
                }
        }
        else
                ;//do nothing
        root.height = max(height(root.left), height(root.right)) + 1;
        return root;
}
```

```
/*Height of tree*/
protected int height(BinaryNode<T> root){
	if(root == null){
		return 0;
	}
	return root.height;
}

/*Return the max*/
protected int max(int a, int b){
	return (a > b) ? a : b;
}

/*Find a match in a tree*/
public boolean find(T id){
	BinarySearchTree<T>.BinaryNode<T> current = root;
	while(current != null){
		if(current.data.compareTo(id) == 0){
			return true;
		}else if(current.data.compareTo(id) == 1){
			current = current.left;
		}else{
			current = current.right;
		}
	}
	return false;
}

/*Print the tree in preorder*/
public String preorder(){
	String str = "\n[Pre-order - size = " + size + "]\n";
	str += preorder(root);
	return str;
}

/*Print the tree in inorder*/
public String inorder(){
	String str = "\n[In-order - size = " + size + "]\n";
	str += inorder(root);
	return str;
}

/*Print the tree in postorder*/
```

```
public String postorder(){
	String str = "\n[Post-order - size = " + size + "]\n";
	str += postorder(root);
	return str;
}

/*Print the tree in preorder*/
protected String preorder(BinaryNode <T> root){
	String msg = "";
	if(root != null){
		msg += root.data + " ";
		msg += preorder(root.left);
		msg += preorder(root.right);
	}
	return msg;
}

/*Print the tree in inorder*/
protected String inorder(BinaryNode <T> root){
	String msg = "";
	if(root != null){
		msg += inorder(root.left);
		msg += root.data + " ";
		msg += inorder(root.right);
	}
	return msg;
}

/*Print the tree in postorder*/
protected String postorder(BinaryNode <T> root){
	String msg = "";
	if(root != null){
		msg += postorder(root.left);
		msg += postorder(root.right);
		msg += root.data + " ";
	}
	return msg;
}

/*Print the tree in level order*/
protected void levelOrderPrint(BinaryNode <T> root){
	Queue<BinaryNode<T>> queue = new LinkedList<BinaryNode<T>>();
	LinkedList<T> list = new LinkedList<T>();
	queue.add(root);
```

```
		while(!queue.isEmpty()){
			BinaryNode<T> item = (BinaryNode<T>) queue.remove();
			if(item != null){
				list.addLast(item.data);
				if(item.left != null)
					queue.add(item.left);
				if(item.right != null)
					queue.add(item.right);
			}
		}
	System.out.println(list);
	}

	/*Insert a level into the tree*/
	protected BinaryNode<T> insertLevel(T d, BinaryNode<T> root){
		if(root == null)
			root = new BinaryNode<T>(d);
		else if(root.left == null){
			root.left = new BinaryNode<T>(d);
			insert(d, root.left);
		}
		else if(root.right == null){
			root.right = new BinaryNode<T>(d);
			insert(d, root.right);
		}
		return root;
	}

	/*Insert a level into the tree*/
	public void insertLevel(T d){
		insertLevel(d, root);
		size++;
	}

	/*Print the tree in level order*/
	public void levelOrderPrint(){
		levelOrderPrint(root);
	}

	/*Level order insert*/
	public void levelOrderInsert(T d) {
size++;
if(root == null)
	root = new BinaryNode<T>(d);
```

```
        else {
            Queue<BinaryNode<T>> queue = new LinkedList<BinaryNode<T>>();

            queue.add(root);
            while(!queue.isEmpty()) {
                BinaryNode<T> item = queue.remove();
                if(item != null) {
                    if(item.left != null){
                        queue.add(item.left);
                        if(item.right != null)
                            queue.add(item.right);
                        else{
                            item.right = new BinaryNode<T>(d);
                            return;
                        }
                    }
                    else{
                        item.left = new BinaryNode<T>(d);
                        return;
                    }
                }
            }
        }
    }
}
```

Sample Output

Starting from a solved cube, when there are 0 moves (12^0 = 1 total outcome),
there was 1 unique case that took 0 moves to solve.
Execution time for this module: 2
Total execution time so far: 2

Starting from a solved cube, when there is 1 move (12^1 = 12 total outcomes),
there were 0 cases that took 0 moves to solve,
6 were unique cases that took 1 move to solve,
while 6 of them were repeat cases that also took 1 move to solve.
Execution time for this module: 2
Total execution time so far: 4

Starting from a solved cube, when there are 2 moves (12^2 = 144 total outcomes),
there were 24 cases that took 0 moves to solve,
there were 0 cases that took 1 move to solve,
27 were unique cases that took 2 moves to solve,
while 93 of them were repeat cases that also took 2 moves to solve.
Execution time for this module: 20
Total execution time so far: 24

Starting from a solved cube, when there are 3 moves (12^3 = 1728 total outcomes),
there were 0 cases that took 0 moves to solve,
there were 576 cases that took 1 move to solve,
there were 0 cases that took 2 moves to solve,
120 were unique cases that took 3 moves to solve,
while 1032 of them were repeat cases that also took 3 moves to solve.
Execution time for this module: 49
Total execution time so far: 73

Starting from a solved cube, when there are 4 moves (12^4 = 20736 total outcomes),
there were 1152 cases that took 0 moves to solve,
there were 0 cases that took 1 move to solve,
there were 8448 cases that took 2 moves to solve,
there were 0 cases that took 3 moves to solve,
534 were unique cases that took 4 moves to solve,
while 10602 of them were repeat cases that also took 4 moves to solve.
Execution time for this module: 160
Total execution time so far: 233

Starting from a solved cube, when there are 5 moves (12^5 = 248832 total outcomes),
there were 0 cases that took 0 moves to solve,
there were 33792 cases that took 1 move to solve,
there were 0 cases that took 2 moves to solve,
there were 107520 cases that took 3 moves to solve,
there were 0 cases that took 4 moves to solve,

2256 were unique cases that took 5 moves to solve,
while 105264 of them were repeat cases that also took 5 moves to solve.
Execution time for this module: 1084
Total execution time so far: 1317

Starting from a solved cube, when there are 6 moves (12^6 = 2985984 total outcomes),
there were 67584 cases that took 0 moves to solve,
there were 0 cases that took 1 move to solve,
there were 586752 cases that took 2 moves to solve,
there were 0 cases that took 3 moves to solve,
there were 1308672 cases that took 4 moves to solve,
there were 0 cases that took 5 moves to solve,
8969 were unique cases that took 6 moves to solve,
while 1014007 of them were repeat cases that also took 6 moves to solve.
Execution time for this module: 12044
Total execution time so far: 13361

Starting from a solved cube, when there are 7 moves (12^7 = 35831808 total outcomes),
there were 0 cases that took 0 moves to solve,
there were 2187264 cases that took 1 move to solve,
there were 0 cases that took 2 moves to solve,
there were 8706048 cases that took 3 moves to solve,
there were 0 cases that took 4 moves to solve,
there were 15470592 cases that took 5 moves to solve,
there were 0 cases that took 6 moves to solve,
33058 were unique cases that took 7 moves to solve,
while 9434846 of them were repeat cases that also took 7 moves to solve.
Execution time for this module: 146790
Total execution time so far: 160151

Starting from a solved cube, when there are 8 moves (12^8 = 429981696 total outcomes),
there were 4374528 cases that took 0 moves to solve,
there were 0 cases that took 1 move to solve,
there were 41914368 cases that took 2 moves to solve,
there were 0 cases that took 3 moves to solve,
there were 122732544 cases that took 4 moves to solve,
there were 0 cases that took 5 moves to solve,
there were 177463296 cases that took 6 moves to solve,
there were 0 cases that took 7 moves to solve,
114149 were unique cases that took 8 moves to solve,
while 83382811 of them were repeat cases that also took 8 moves to solve.
Execution time for this module: 1838482
Total execution time so far: 1998633

Starting from a solved cube, when there are 9 moves (12^9 = 5159780352 total outcomes),
there were 0 cases that took 0 moves to solve,
there were 150331392 cases that took 1 move to solve,

there were 0 cases that took 2 moves to solve,
there were 688570368 cases that took 3 moves to solve,
there were 0 cases that took 4 moves to solve,
there were 1673650176 cases that took 5 moves to solve,
there were 0 cases that took 6 moves to solve,
there were 1934032896 cases that took 7 moves to solve,
there were 0 cases that took 8 moves to solve,
360508 were unique cases that took 9 moves to solve,
while 712835012 of them were repeat cases that also took 9 moves to solve.
Execution time for this module: 22913376
Total execution time so far: 24912009

Starting from a solved cube, when there are 10 moves (12^10 = 61917364224 total outcomes),
there were 300662784 cases that took 0 moves to solve,
there were 0 cases that took 1 move to solve,
there were 3082223616 cases that took 2 moves to solve,
there were 0 cases that took 3 moves to solve,
there were 10815553536 cases that took 4 moves to solve,
there were 0 cases that took 5 moves to solve,
there were 22057844736 cases that took 6 moves to solve,
there were 0 cases that took 7 moves to solve,
there were 19904888832 cases that took 8 moves to solve,
there were 0 cases that took 9 moves to solve,
930588 were unique cases that took 10 moves to solve,
while 5755260132 of them were repeat cases that also took 10 moves to solve.
Execution time for this module: 286919452
Total execution time so far: 311831461

Starting from a solved cube, when there are 11 moves (12^11 = 743008370688 total outcomes),
there were 0 cases that took 0 moves to solve,
there were 10777264128 cases that took 1 move to solve,
there were 0 cases that took 2 moves to solve,
there were 54672457728 cases that took 3 moves to solve,
there were 0 cases that took 4 moves to solve,
there were 164831232000 cases that took 5 moves to solve,
there were 0 cases that took 6 moves to solve,
there were 272987848704 cases that took 7 moves to solve,
there were 0 cases that took 8 moves to solve,
there were 198771081216 cases that took 9 moves to solve,
there were 0 cases that took 10 moves to solve,
1350852 were unique cases that took 11 moves to solve,
while 40967136060 of them were repeat cases that also took 11 moves to solve.
Execution time for this module: 3591826202
Total execution time so far: 3903657663

Appendix C

CubeCyclesTest

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: CubeCyclesTest
 * Class: Test
 * Purpose of this Program: The purpose of this program is to systematically
 * go through every possible sequence a 3x3x3 Rubik's cube can exist in,
 * up to distance 10, and output all outcomes to a .txt file that are cycles
 *  with a total amount of moves that is less than or equal to 20.
 *  */

package cube;

import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.PrintStream;

public class Test {

	static OrientationMethods methods = new OrientationMethods();
	static String[] sequence = new String[40];

	/*  intakes the I value and performs the appropriate
	 *  orientation method on the array/cube
	 */
	public static void switcher(int I, int[] array, int order){
		switch(i){
		case 1: methods.left(array);
		break;
		case 2: methods.leftPrime(array);
		break;
		case 3: methods.right(array);
		break;
		case 4: methods.rightPrime(array);
		break;
		case 5: methods.up(array);
		break;
		case 6: methods.upPrime(array);
		break;
		case 7: methods.down(array);
		break;
		case 8: methods.downPrime(array);
```

```
        break;
        case 9: methods.back(array);
        break;
        case 10: methods.backPrime(array);
        break;
        case 11: methods.front(array);
        break;
        case 12: methods.frontPrime(array);
        break;
        case 13: methods.e(array);
        break;
        case 14: methods.ePrime(array);
        break;
        case 15: methods.m(array);
        break;
        case 16: methods.mPrime(array);
        break;
        case 17: methods.s(array);
        break;
        case 18: methods.sPrime(array);
        break;
        default: break;
        }
}

public static void converter(int I, int j){
        switch(i){
        case 1: sequence[j] = “L”;
        break;
        case 2: sequence[j] = “L’”;
        break;
        case 3: sequence[j] = “R”;
        break;
        case 4: sequence[j] = “R’”;
        break;
        case 5: sequence[j] = “U”;
        break;
        case 6: sequence[j] = “U’”;
        break;
        case 7: sequence[j] = “D”;
        break;
        case 8: sequence[j] = “D’”;
        break;
        case 9: sequence[j] = “B”;
```

```
            break;
            case 10: sequence[j] = “B’”;
            break;
            case 11: sequence[j] = “F”;
            break;
            case 12: sequence[j] = “F’”;
            break;
            case 13: sequence[j] = “E”;
            break;
            case 14: sequence[j] = “E’”;
            break;
            case 15: sequence[j] = “M”;
            break;
            case 16: sequence[j] = “M’”;
            break;
            case 17: sequence[j] = “S”;
            break;
            case 18: sequence[j] = “S’”;
            break;
            default: break;
            }
}

public static boolean comparator(int[] array1, int[] array2){
            for(int I = 0; I < 54; i++){
                        if(array1[i] != array2[i]){
                                    return false;
                        }
            }
            return true;
}

public static void main(String[] args) throws FileNotFoundException{
            File file = new File(“results.txt”);
            FileOutputStream fos = new FileOutputStream(file);
            PrintStream ps = new PrintStream(fos);
            System.setOut(ps);
            int counter = 0;
            boolean found = false;
            int[] solvedCube = {0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,
                                                1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,
                                                4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
            int[] testCube = {0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,
                                            1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,
```

```
                                        4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
for(int I = 1; I < 19; i++){
        int[] oneCube = {0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,
                                        1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,
                                        4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
        for(int j = 0; j < 20; j++){
                switcher(I, oneCube, 0);
                testCube = oneCube;
                methods.reorient(testCube);
                if(comparator(testCube, solvedCube)){
                        if(found == false){
                                counter++;
                                System.out.print(counter + ". Sequence: ");
                                converter(I, 0);
                                for(int a = 0; a < 1; a++){
                                        System.out.print(sequence[a] + " – ");
                                }
                                found = true;
                        }
                        System.out.print((j + 1) + " times ");
                }
        }
        if(found == true){
                System.out.println("");
        }
        found = false;
}
counter = 0;
System.out.println("==============");
for(int I = 1; I < 19; i++){
        for(int j = 1; j < 19; j++){
                int[] twoCube = {0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,
                                        1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,
                                        4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
                for(int k = 0; k < 10; k++){
                        switcher(I, twoCube, 0);
                        switcher(j, twoCube, 1);
                        testCube = twoCube;
                        methods.reorient(testCube);
                        if(comparator(testCube, solvedCube)){
                                if(found == false){
                                        counter++;
                                        System.out.print(counter + ". Sequence: ");
                                        converter(I, 0);
```

```
                                        converter(j, 1);
                                        for(int a = 0; a < 2; a++){
                                                System.out.print(sequence[a]);
                                        }
                                        System.out.print(" – ");
                                        found = true;
                                }
                                System.out.print((k + 1) + " times ");
                        }//closes the comparator
                }//closes the 20 loop
                if(found == true){
                        System.out.println("");
                }
                found = false;
        }
}
counter = 0;
System.out.println("==============");
for(int I = 1; I < 19; i++){
        for(int j = 1; j < 19; j++){
                for(int k = 1; k < 19; k++){
                        int[] threeCube = {0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,

1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,

4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
                        for(int l = 0; l < 6; l++){
                                switcher(I, threeCube, 0);
                                switcher(j, threeCube, 1);
                                switcher(k, threeCube, 2);
                                testCube = threeCube;
                                methods.reorient(testCube);
                                if(comparator(testCube, solvedCube)){
                                        if(found == false){
                                                counter++;
                                                System.out.print(counter + ".
Sequence: ");
                                                converter(I, 0);
                                                converter(j, 1);
                                                converter(k, 2);
                                                for(int a = 0; a < 3; a++){

        System.out.print(sequence[a]);
                                                }
```

```
                                                System.out.print(" – ");
                                                found = true;
                                        }
                                        System.out.print((l + 1) + " times ");
                                }//closes the comparator
                        }//closes the 20 loop
                        if(found == true){
                                System.out.println("");
                        }
                        found = false;
                }
        }
}
counter = 0;
System.out.println("==============");
for(int I = 1; I < 19; i++){
        for(int j = 1; j < 19; j++){
                for(int k = 1; k < 19; k++){
                        for(int l = 1; l < 19; l++){
                                int[] fourCube = {0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,

1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,

4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
                                for(int m = 0; m < 5; m++){
                                        switcher(I, fourCube, 0);
                                        switcher(j, fourCube, 1);
                                        switcher(k, fourCube, 2);
                                        switcher(l, fourCube, 3);
                                        testCube = fourCube;
                                        methods.reorient(testCube);
                                        if(comparator(testCube, solvedCube)){
                                                if(found == false){
                                                        counter++;
                                                        System.out.print(counter + ".
Sequence: ");

                                                        converter(I, 0);
                                                        converter(j, 1);
                                                        converter(k, 2);
                                                        converter(l, 3);
                                                        for(int a = 0; a < 4; a++){

        System.out.print(sequence[a]);
                                                        }
```

```
                                                                System.out.print(" – ");
                                                                found = true;
                                                        }
                                                        System.out.print((m + 1) + " times
");
                                                }//closes the comparator
                                        }//closes the 20 loop
                                        if(found == true){
                                                System.out.println("");
                                        }
                                        found = false;
                                }
                        }
                }
        }
        counter = 0;
        System.out.println("==============");
        for(int I = 1; I < 19; i++){
                for(int j = 1; j < 19; j++){
                        for(int k = 1; k < 19; k++){
                                for(int l = 1; l < 19; l++){
                                        for(int m = 1; m < 19; m++){
                                                int[] fiveCube =
{0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,

1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,

4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
                                                for(int n = 0; n < 4; n++){
                                                        switcher(I, fiveCube, 0);
                                                        switcher(j, fiveCube, 1);
                                                        switcher(k, fiveCube, 2);
                                                        switcher(l, fiveCube, 3);
                                                        switcher(m, fiveCube, 4);
                                                        testCube = fiveCube;
                                                        methods.reorient(testCube);
                                                        if(comparator(testCube,
solvedCube)){
                                                                if(found == false){
                                                                        counter++;

        System.out.print(counter + ". Sequence: ");
                                                                        converter(I, 0);
                                                                        converter(j, 1);
```

```
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                converter(k, 2);
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                converter(l, 3);
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                converter(m, 4);
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                for(int a = 0; a < 5;
a++){

                System.out.print(sequence[a]);
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
```

```
                                                                        switcher(m, sixCube, 4);
                                                                        switcher(n, sixCube, 5);
                                                                        testCube = sixCube;
                                                                        methods.reorient(testCube);
                                                                        if(comparator(testCube,
solvedCube)){
                                                                                if(found == false){
                                                                                        counter++;

        System.out.print(counter + ". Sequence: ");
                                                                                        converter(l, 0);
                                                                                        converter(j, 1);
                                                                                        converter(k, 2);
                                                                                        converter(l, 3);
                                                                                        converter(m, 4);
                                                                                        converter(n, 5);
                                                                                        for(int a = 0; a < 6;
a++){

        System.out.print(sequence[a]);
                                                                                        }
                                                                                        System.out.print(" –
");
                                                                                        found = true;
                                                                                }
                                                                                System.out.print((o + 1) + "
times ");
                                                                        }//closes the comparator
                                                                }//closes the 20 loop
                                                                if(found == true){
                                                                        System.out.println("");
                                                                }
                                                                found = false;
                                                                }
                                                        }
                                                }
                                        }
                                }
                        }
                        counter = 0;
                        System.out.println("==============");
                        for(int I = 1; I < 19; i++){
                                for(int j = 1; j < 19; j++){
                                        for(int k = 1; k < 19; k++){
```

```
for(int l = 1; l < 19; l++){
for(int m = 1; m < 19; m++){
for(int n = 1; n < 19; n++){
for(int o = 1; o < 19; o++){
int[] sevenCube =
{0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,

1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,

4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
for(int p = 0; p < 2; p++){
switcher(l,
sevenCube, 0);
switcher(j,
sevenCube, 1);
switcher(k,
sevenCube, 2);
switcher(l,
sevenCube, 3);
switcher(m,
sevenCube, 4);
switcher(n,
sevenCube, 5);
switcher(o,
sevenCube, 6);
testCube =
sevenCube;

methods.reorient(testCube);

if(comparator(testCube, solvedCube)){
if(found ==
false){

counter++;

System.out.print(counter + ". Sequence: ");

converter(l, 0);

converter(j, 1);

converter(k, 2);
```

```
converter(l, 3);

converter(m, 4);

converter(n, 5);

converter(o, 6);
                                                
```

```
for(int p = 1; p < 19; p++){
int[] eightCube =
{0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,

1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,

4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
for(int q = 0; q < 2;
q++){
switcher(I,
eightCube, 0);
switcher(j,
eightCube, 1);
switcher(k,
eightCube, 2);
switcher(l,
eightCube, 3);
switcher(m,
eightCube, 4);
switcher(n,
eightCube, 5);
switcher(o,
eightCube, 6);
switcher(p,
eightCube, 7);
testCube =
eightCube;

methods.reorient(testCube);

if(comparator(testCube, solvedCube)){

if(found == false){

counter++;

System.out.print(counter + ". Sequence: ");

converter(I, 0);

converter(j, 1);

converter(k, 2);
```

```
converter(l, 3);

converter(m, 4);

converter(n, 5);

converter(o, 6);

converter(p, 7);

for(int a = 0; a < 8; a++){

System.out.print(sequence[a]);

}

System.out.print(" – ");

found = true;
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}

System.out.print((q + 1) + " times ");
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}//closes the comparator
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}//closes the 20 loop
if(found == true){

System.out.println("");
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
found = false;
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
}
}
}
}
}
}
}
}
counter = 0;
System.out.println("==============");
for(int I = 1; I < 19; i++){
for(int j = 1; j < 19; j++){
for(int k = 1; k < 19; k++){
```

```
for(int l = 1; l < 19; l++){
for(int m = 1; m < 19; m++){
for(int n = 1; n < 19; n++){
for(int o = 1; o < 19; o++){
for(int p = 1; p < 19; p++){
for(int q = 1; q < 19;
q++){
int[] nineCube
= {0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,

1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,

4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
for(int r = 0; r
< 2; r++){

switcher(I, nineCube, 0);

switcher(j, nineCube, 1);

switcher(k, nineCube, 2);

switcher(l, nineCube, 3);

switcher(m, nineCube, 4);

switcher(n, nineCube, 5);

switcher(o, nineCube, 6);

switcher(p, nineCube, 7);

switcher(q, nineCube, 8);

testCube = nineCube;

methods.reorient(testCube);

if(comparator(testCube, solvedCube)){

if(found == false){

counter++;
```

```
System.out.print(counter + ". Sequence: ");
converter(I, 0);
converter(j, 1);
converter(k, 2);
converter(l, 3);
converter(m, 4);
converter(n, 5);
converter(o, 6);
converter(p, 7);
converter(q, 8);
for(int a = 0; a < 9; a++){
        System.out.print(sequence[a]);
}
System.out.print(" – ");
found = true;
}
System.out.print((r + 1) + " times ");
}//closes the comparator
}//closes the 20 loop
if(found == true){
System.out.println("");
}
found = false;
```

```
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
}
}
}
}
}
}
}
}
counter = 0;
System.out.println("==============");
for(int I = 1; I < 19; i++){
for(int j = 1; j < 19; j++){
for(int k = 1; k < 19; k++){
for(int l = 1; l < 19; l++){
for(int m = 1; m < 19; m++){
for(int n = 1; n < 19; n++){
for(int o = 1; o < 19; o++){
for(int p = 1; p < 19; p++){
for(int q = 1; q < 19;
q++){
for(int r = 1; r
< 19; r++){
int[]
tenCube = {0,0,0,0,0,0,0,0,0,1,1,1,2,2,2,3,3,3,

1,1,1,2,2,2,3,3,3,1,1,1,2,2,2,3,3,3,

4,4,4,4,4,4,4,4,4,5,5,5,5,5,5,5,5,5};
for(int
s = 0; s < 2; s++){

switcher(I, tenCube, 0);

switcher(j, tenCube, 1);

switcher(k, tenCube, 2);

switcher(l, tenCube, 3);

switcher(m, tenCube, 4);

switcher(n, tenCube, 5);
```

```
switcher(o, tenCube, 6);

switcher(p, tenCube, 7);

switcher(q, tenCube, 8);

switcher(r, tenCube, 9);

testCube = tenCube;

methods.reorient(testCube);

if(comparator(testCube, solvedCube)){

if(found == false){

        counter++;

        System.out.print(counter + ". Sequence: ");

        converter(I, 0);

        converter(j, 1);

        converter(k, 2);

        converter(l, 3);

        converter(m, 4);

        converter(n, 5);

        converter(o, 6);

        converter(p, 7);

        converter(q, 8);

        converter(r, 9);

        for(int a = 0; a < 10; a++){

                System.out.print(sequence[a]);
```

```
                }

                System.out.print(" – ");

                found = true;

        }

        System.out.print((s + 1) + " times ");

        }//closes the comparator

        }//closes the 20 loop

        if(found == true){

        System.out.println("");
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
found
= false;
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
                System.out.print(" – ");
                found = true;
        }
        System.out.print((s + 1) + " times ");
        }//closes the comparator
        }//closes the 20 loop
        if(found == true){
        System.out.println("");
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
                                                                                                                                                                                                                                                                                                                                                                                                                                                
}
                                                                                                                                                                                                
}
                                                
}
                
}
                counter = 0;
                System.out.println("==============");
        }
}
```

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: CubeCyclesTest
 * Class: OrientationMethods
 * Purpose of this Program: The purpose of this program is to systematically
 * go through every possible sequence a 3x3x3 Rubik's cube can exist in,
 * up to distance 10, and output all outcomes to a .txt file that are cycles
 *  with a total amount of moves that is less than or equal to 20.
 *  */

package cube;

public class OrientationMethods {

	public void left(int[] array){//performs a left slice clock-wise quarter turn.
		Int temp = array[0];
		array[0] = array[45];
		array[45] = array[36];
		array[36] = array[12];
		array[12] = temp;

		temp = array[3];
		array[3] = array[48];
		array[48] = array[39];
		array[39] = array[21];
		array[21] = temp;

		temp = array[6];
		array[6] = array[51];
		array[51] = array[42];
		array[42] = array[30];
		array[30] = temp;

		temp = array[9];
		array[9] = array[27];
		array[27] = array[29];
		array[29] = array[11];
		array[11] = temp;

		temp = array[10];
		array[10] = array[18];
		array[18] = array[28];
```

```
        array[28] = array[20];
        array[20] = temp;
}

public void leftPrime(int[] array){//performs a left slice counter-clock-wise quarter turn.
        Int temp = array[0];
        array[0] = array[12];
        array[12] = array[36];
        array[36] = array[45];
        array[45] = temp;

        temp = array[3];
        array[3] = array[21];
        array[21] = array[39];
        array[39] = array[48];
        array[48] = temp;

        temp = array[6];
        array[6] = array[30];
        array[30] = array[42];
        array[42] = array[51];
        array[51] = temp;

        temp = array[9];
        array[9] = array[11];
        array[11] = array[29];
        array[29] = array[27];
        array[27] = temp;

        temp = array[10];
        array[10] = array[20];
        array[20] = array[28];
        array[28] = array[18];
        array[18] = temp;
}

public void right(int[] array){//performs a right slice clock-wise quarter turn.
        Int temp = array[53];
        array[53] = array[8];
        array[8] = array[32];
        array[32] = array[44];
        array[44] = temp;

        temp = array[50];
```

```
            array[50] = array[5];
            array[5] = array[23];
            array[23] = array[41];
            array[41] = temp;

            temp = array[47];
            array[47] = array[2];
            array[2] = array[14];
            array[14] = array[38];
            array[38] = temp;

            temp = array[33];
            array[33] = array[35];
            array[35] = array[17];
            array[17] = array[15];
            array[15] = temp;

            temp = array[24];
            array[24] = array[34];
            array[34] = array[26];
            array[26] = array[16];
            array[16] = temp;
        }

        public void rightPrime(int[] array){//performs a right slice counter-clock-wise quarter
turn.
            Int temp = array[53];
            array[53] = array[44];
            array[44] = array[32];
            array[32] = array[8];
            array[8] = temp;

            temp = array[50];
            array[50] = array[41];
            array[41] = array[23];
            array[23] = array[5];
            array[5] = temp;

            temp = array[47];
            array[47] = array[38];
            array[38] = array[14];
            array[14] = array[2];
            array[2] = temp;
```

```
        temp = array[33];
        array[33] = array[15];
        array[15] = array[17];
        array[17] = array[35];
        array[35] = temp;

        temp = array[24];
        array[24] = array[16];
        array[16] = array[26];
        array[26] = array[34];
        array[34] = temp;
}

public void up(int[] array){//performs an up slice clock-wise quarter turn.
        Int temp = array[6];
        array[6] = array[29];
        array[29] = array[38];
        array[38] = array[15];
        array[15] = temp;

        temp = array[7];
        array[7] = array[20];
        array[20] = array[37];
        array[37] = array[24];
        array[24] = temp;

        temp = array[8];
        array[8] = array[11];
        array[11] = array[36];
        array[36] = array[33];
        array[33] = temp;

        temp = array[12];
        array[12] = array[30];
        array[30] = array[32];
        array[32] = array[14];
        array[14] = temp;

        temp = array[13];
        array[13] = array[21];
        array[21] = array[31];
        array[31] = array[23];
        array[23] = temp;
}
```

```
public void upPrime(int[] array){//performs an up slice counter-clock-wise quarter turn.
        Int temp = array[6];
        array[6] = array[15];
        array[15] = array[38];
        array[38] = array[29];
        array[29] = temp;

        temp = array[7];
        array[7] = array[24];
        array[24] = array[37];
        array[37] = array[20];
        array[20] = temp;

        temp = array[8];
        array[8] = array[33];
        array[33] = array[36];
        array[36] = array[11];
        array[11] = temp;

        temp = array[12];
        array[12] = array[14];
        array[14] = array[32];
        array[32] = array[30];
        array[30] = temp;

        temp = array[13];
        array[13] = array[23];
        array[23] = array[31];
        array[31] = array[21];
        array[21] = temp;
}

public void down(int[] array){//performs a down slice clock-wise quarter turn.
        Int temp = array[0];
        array[0] = array[17];
        array[17] = array[44];
        array[44] = array[27];
        array[27] = temp;

        temp = array[1];
        array[1] = array[26];
        array[26] = array[43];
        array[43] = array[18];
```

```
            array[18] = temp;

            temp = array[2];
            array[2] = array[35];
            array[35] = array[42];
            array[42] = array[9];
            array[9] = temp;

            temp = array[45];
            array[45] = array[51];
            array[51] = array[53];
            array[53] = array[47];
            array[47] = temp;

            temp = array[46];
            array[46] = array[48];
            array[48] = array[52];
            array[52] = array[50];
            array[50] = temp;
        }

        public void downPrime(int[] array){//performs a down slice counter-clock-wise quarter
turn.
            Int temp = array[0];
            array[0] = array[27];
            array[27] = array[44];
            array[44] = array[17];
            array[17] = temp;

            temp = array[1];
            array[1] = array[18];
            array[18] = array[43];
            array[43] = array[26];
            array[26] = temp;

            temp = array[2];
            array[2] = array[9];
            array[9] = array[42];
            array[42] = array[35];
            array[35] = temp;

            temp = array[45];
            array[45] = array[47];
            array[47] = array[53];
```

```
        array[53] = array[51];
        array[51] = temp;

        temp = array[46];
        array[46] = array[50];
        array[50] = array[52];
        array[52] = array[48];
        array[48] = temp;
    }

    public void front(int[] array){//performs a front slice clock-wise quarter turn.
        Int temp = array[27];
        array[27] = array[47];
        array[47] = array[33];
        array[33] = array[30];
        array[30] = temp;

        temp = array[28];
        array[28] = array[46];
        array[46] = array[34];
        array[34] = array[31];
        array[31] = temp;

        temp = array[29];
        array[29] = array[45];
        array[45] = array[35];
        array[35] = array[32];
        array[32] = temp;

        temp = array[36];
        array[36] = array[42];
        array[42] = array[44];
        array[44] = array[38];
        array[38] = temp;

        temp = array[37];
        array[37] = array[39];
        array[39] = array[43];
        array[43] = array[41];
        array[41] = temp;
    }

    public void frontPrime(int[] array){//performs a front slice counter-clock-wise quarter
turn.
```

```
        Int temp = array[27];
        array[27] = array[30];
        array[30] = array[33];
        array[33] = array[47];
        array[47] = temp;

        temp = array[28];
        array[28] = array[31];
        array[31] = array[34];
        array[34] = array[46];
        array[46] = temp;

        temp = array[29];
        array[29] = array[32];
        array[32] = array[35];
        array[35] = array[45];
        array[45] = temp;

        temp = array[36];
        array[36] = array[38];
        array[38] = array[44];
        array[44] = array[42];
        array[42] = temp;

        temp = array[37];
        array[37] = array[41];
        array[41] = array[43];
        array[43] = array[39];
        array[39] = temp;
}

public void back(int[] array){//performs a back slice clock-wise quarter turn.
        Int temp = array[9];
        array[9] = array[12];
        array[12] = array[15];
        array[15] = array[53];
        array[53] = temp;

        temp = array[10];
        array[10] = array[13];
        array[13] = array[16];
        array[16] = array[52];
        array[52] = temp;
```

```
            temp = array[11];
            array[11] = array[14];
            array[14] = array[17];
            array[17] = array[51];
            array[51] = temp;

            temp = array[0];
            array[0] = array[6];
            array[6] = array[8];
            array[8] = array[2];
            array[2] = temp;

            temp = array[1];
            array[1] = array[3];
            array[3] = array[7];
            array[7] = array[5];
            array[5] = temp;
        }

        public void backPrime(int[] array){//performs a back slice counter-clock-wise quarter
turn.
            Int temp = array[9];
            array[9] = array[53];
            array[53] = array[15];
            array[15] = array[12];
            array[12] = temp;

            temp = array[10];
            array[10] = array[52];
            array[52] = array[16];
            array[16] = array[13];
            array[13] = temp;

            temp = array[11];
            array[11] = array[51];
            array[51] = array[17];
            array[17] = array[14];
            array[14] = temp;

            temp = array[0];
            array[0] = array[2];
            array[2] = array[8];
            array[8] = array[6];
            array[6] = temp;
```

```
            temp = array[1];
            array[1] = array[5];
            array[5] = array[7];
            array[7] = array[3];
            array[3] = temp;
        }

        public void m(int[] array){//performs a middle layer (between L and R) clock-wise(as L)
quarter turn.
            Int temp = array[1];
            array[1] = array[46];
            array[46] = array[37];
            array[37] = array[13];
            array[13] = temp;

            temp = array[4];
            array[4] = array[49];
            array[49] = array[40];
            array[40] = array[22];
            array[22] = temp;

            temp = array[7];
            array[7] = array[52];
            array[52] = array[43];
            array[43] = array[31];
            array[31] = temp;
        }

        public void mPrime(int[] array){//performs a middle layer(between L and R) counter-
clock-wise(as L) quarter turn.
            Int temp = array[1];
            array[1] = array[13];
            array[13] = array[37];
            array[37] = array[46];
            array[46] = temp;

            temp = array[4];
            array[4] = array[22];
            array[22] = array[40];
            array[40] = array[49];
            array[49] = temp;

            temp = array[7];
```

```
            array[7] = array[31];
            array[31] = array[43];
            array[43] = array[52];
            array[52] = temp;
        }

        public void s(int[] array){//performs a middle layer(between F and B) clock-wise(as F)
quarter turn.
            Int temp = array[18];
            array[18] = array[50];
            array[50] = array[24];
            array[24] = array[21];
            array[21] = temp;

            temp = array[19];
            array[19] = array[49];
            array[49] = array[25];
            array[25] = array[22];
            array[22] = temp;

            temp = array[20];
            array[20] = array[48];
            array[48] = array[26];
            array[26] = array[23];
            array[23] = temp;
        }

        public void sPrime(int[] array){//performs a middle layer(between F and B) counter-
clock-wise(as F) quarter turn.
            Int temp = array[18];
            array[18] = array[21];
            array[21] = array[24];
            array[24] = array[50];
            array[50] = temp;

            temp = array[19];
            array[19] = array[22];
            array[22] = array[25];
            array[25] = array[49];
            array[49] = temp;

            temp = array[20];
            array[20] = array[23];
            array[23] = array[26];
```

```
            array[26] = array[48];
            array[48] = temp;
        }

        public void e(int[] array){//performs a middle layer(between U and D) clock-wise(as D)
quarter turn.
            Int temp = array[10];
            array[10] = array[5];
            array[5] = array[34];
            array[34] = array[39];
            array[39] = temp;

            temp = array[19];
            array[19] = array[4];
            array[4] = array[25];
            array[25] = array[40];
            array[40] = temp;

            temp = array[28];
            array[28] = array[3];
            array[3] = array[16];
            array[16] = array[41];
            array[41] = temp;
        }

        public void ePrime(int[] array){//performs a middle layer(between U and D) counter-
clock-wise(as D) quarter turn.
            Int temp = array[10];
            array[10] = array[39];
            array[39] = array[34];
            array[34] = array[5];
            array[5] = temp;

            temp = array[19];
            array[19] = array[40];
            array[40] = array[25];
            array[25] = array[4];
            array[4] = temp;

            temp = array[28];
            array[28] = array[41];
            array[41] = array[16];
            array[16] = array[3];
            array[3] = temp;
```

```
}

public void x(int[] array){//rotate the entire cube clockwise on R
	leftPrime(array);
	mPrime(array);
	right(array);
}

public void xPrime(int[] array){//rotate the entire cube counter-clockwise on R
	left(array);
	m(array);
	rightPrime(array);
}

public void y(int[] array){//rotate the entire cube clockwise on U
	up(array);
	ePrime(array);
	downPrime(array);
}

public void yPrime(int[] array){//rotate the entire cube counter-clockwise on U
	upPrime(array);
	e(array);
	down(array);
}

public void z(int[] array){//rotate the entire cube clockwise on F
	front(array);
	s(array);
	backPrime(array);
}

public void zPrime(int[] array){//rotate the entire cube counter-clockwise on F
	frontPrime(array);
	sPrime(array);
	back(array);
}

public void reorient(int[] array){//orient the cube to standard form
	while(array[4] != 0 || array[19] != 1){
		while(array[4] != 0){
			if(array[19] != 0 && array[25] != 0){
				x(array);
			}
```

```
                    else{
                        y(array);
                    }
                }
                while(array[19] != 1){
                    z(array);
                }
            }
        }
}
```

Sample Output

1. Sequence: L - 4 times 8 times 12 times 16 times 20 times
2. Sequence: L' - 4 times 8 times 12 times 16 times 20 times
3. Sequence: R - 4 times 8 times 12 times 16 times 20 times
4. Sequence: R' - 4 times 8 times 12 times 16 times 20 times
5. Sequence: U - 4 times 8 times 12 times 16 times 20 times
6. Sequence: U' - 4 times 8 times 12 times 16 times 20 times
7. Sequence: D - 4 times 8 times 12 times 16 times 20 times
8. Sequence: D' - 4 times 8 times 12 times 16 times 20 times
9. Sequence: B - 4 times 8 times 12 times 16 times 20 times
10. Sequence: B' - 4 times 8 times 12 times 16 times 20 times
11. Sequence: F - 4 times 8 times 12 times 16 times 20 times
12. Sequence: F' - 4 times 8 times 12 times 16 times 20 times
13. Sequence: E - 4 times 8 times 12 times 16 times 20 times
14. Sequence: E' - 4 times 8 times 12 times 16 times 20 times
15. Sequence: M - 4 times 8 times 12 times 16 times 20 times
16. Sequence: M' - 4 times 8 times 12 times 16 times 20 times
17. Sequence: S - 4 times 8 times 12 times 16 times 20 times
18. Sequence: S' - 4 times 8 times 12 times 16 times 20 times

==========

1. Sequence: LL - 2 times 4 times 6 times 8 times 10 times
2. Sequence: LL' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
3. Sequence: LR - 4 times 8 times
4. Sequence: LR' - 4 times 8 times
5. Sequence: LM - 4 times 8 times
6. Sequence: LM' - 4 times 8 times
7. Sequence: L'L - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
8. Sequence: L'L' - 2 times 4 times 6 times 8 times 10 times
9. Sequence: L'R - 4 times 8 times
10. Sequence: L'R' - 4 times 8 times
11. Sequence: L'M - 4 times 8 times
12. Sequence: L'M' - 4 times 8 times
13. Sequence: RL - 4 times 8 times
14. Sequence: RL' - 4 times 8 times
15. Sequence: RR - 2 times 4 times 6 times 8 times 10 times
16. Sequence: RR' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
17. Sequence: RM - 4 times 8 times
18. Sequence: RM' - 4 times 8 times
19. Sequence: R'L - 4 times 8 times

20. Sequence: R'L' - 4 times 8 times
21. Sequence: R'R - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
22. Sequence: R'R' - 2 times 4 times 6 times 8 times 10 times
23. Sequence: R'M - 4 times 8 times
24. Sequence: R'M' - 4 times 8 times
25. Sequence: UU - 2 times 4 times 6 times 8 times 10 times
26. Sequence: UU' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
27. Sequence: UD - 4 times 8 times
28. Sequence: UD' - 4 times 8 times
29. Sequence: UE - 4 times 8 times
30. Sequence: UE' - 4 times 8 times
31. Sequence: U'U - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
32. Sequence: U'U' - 2 times 4 times 6 times 8 times 10 times
33. Sequence: U'D - 4 times 8 times
34. Sequence: U'D' - 4 times 8 times
35. Sequence: U'E - 4 times 8 times
36. Sequence: U'E' - 4 times 8 times
37. Sequence: DU - 4 times 8 times
38. Sequence: DU' - 4 times 8 times
39. Sequence: DD - 2 times 4 times 6 times 8 times 10 times
40. Sequence: DD' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
41. Sequence: DE - 4 times 8 times
42. Sequence: DE' - 4 times 8 times
43. Sequence: D'U - 4 times 8 times
44. Sequence: D'U' - 4 times 8 times
45. Sequence: D'D - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
46. Sequence: D'D' - 2 times 4 times 6 times 8 times 10 times
47. Sequence: D'E - 4 times 8 times
48. Sequence: D'E' - 4 times 8 times
49. Sequence: BB - 2 times 4 times 6 times 8 times 10 times
50. Sequence: BB' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
51. Sequence: BF - 4 times 8 times
52. Sequence: BF' - 4 times 8 times
53. Sequence: BS - 4 times 8 times
54. Sequence: BS' - 4 times 8 times
55. Sequence: B'B - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
56. Sequence: B'B' - 2 times 4 times 6 times 8 times 10 times

57. Sequence: B'F - 4 times 8 times
58. Sequence: B'F' - 4 times 8 times
59. Sequence: B'S - 4 times 8 times
60. Sequence: B'S' - 4 times 8 times
61. Sequence: FB - 4 times 8 times
62. Sequence: FB' - 4 times 8 times
63. Sequence: FF - 2 times 4 times 6 times 8 times 10 times
64. Sequence: FF' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
65. Sequence: FS - 4 times 8 times
66. Sequence: FS' - 4 times 8 times
67. Sequence: F'B - 4 times 8 times
68. Sequence: F'B' - 4 times 8 times
69. Sequence: F'F - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
70. Sequence: F'F' - 2 times 4 times 6 times 8 times 10 times
71. Sequence: F'S - 4 times 8 times
72. Sequence: F'S' - 4 times 8 times
73. Sequence: EU - 4 times 8 times
74. Sequence: EU' - 4 times 8 times
75. Sequence: ED - 4 times 8 times
76. Sequence: ED' - 4 times 8 times
77. Sequence: EE - 2 times 4 times 6 times 8 times 10 times
78. Sequence: EE' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
79. Sequence: EM - 6 times
80. Sequence: EM' - 6 times
81. Sequence: ES - 6 times
82. Sequence: ES' - 6 times
83. Sequence: E'U - 4 times 8 times
84. Sequence: E'U' - 4 times 8 times
85. Sequence: E'D - 4 times 8 times
86. Sequence: E'D' - 4 times 8 times
87. Sequence: E'E - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
88. Sequence: E'E' - 2 times 4 times 6 times 8 times 10 times
89. Sequence: E'M - 6 times
90. Sequence: E'M' - 6 times
91. Sequence: E'S - 6 times
92. Sequence: E'S' - 6 times
93. Sequence: ML - 4 times 8 times
94. Sequence: ML' - 4 times 8 times
95. Sequence: MR - 4 times 8 times
96. Sequence: MR' - 4 times 8 times

97. Sequence: ME - 6 times
98. Sequence: ME' - 6 times
99. Sequence: MM - 2 times 4 times 6 times 8 times 10 times
100. Sequence: MM' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
101. Sequence: MS - 6 times
102. Sequence: MS' - 6 times
103. Sequence: M'L - 4 times 8 times
104. Sequence: M'L' - 4 times 8 times
105. Sequence: M'R - 4 times 8 times
106. Sequence: M'R' - 4 times 8 times
107. Sequence: M'E - 6 times
108. Sequence: M'E' - 6 times
109. Sequence: M'M - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
110. Sequence: M'M' - 2 times 4 times 6 times 8 times 10 times
111. Sequence: M'S - 6 times
112. Sequence: M'S' - 6 times
113. Sequence: SB - 4 times 8 times
114. Sequence: SB' - 4 times 8 times
115. Sequence: SF - 4 times 8 times
116. Sequence: SF' - 4 times 8 times
117. Sequence: SE - 6 times
118. Sequence: SE' - 6 times
119. Sequence: SM - 6 times
120. Sequence: SM' - 6 times
121. Sequence: SS - 2 times 4 times 6 times 8 times 10 times
122. Sequence: SS' - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
123. Sequence: S'B - 4 times 8 times
124. Sequence: S'B' - 4 times 8 times
125. Sequence: S'F - 4 times 8 times
126. Sequence: S'F' - 4 times 8 times
127. Sequence: S'E - 6 times
128. Sequence: S'E' - 6 times
129. Sequence: S'M - 6 times
130. Sequence: S'M' - 6 times
131. Sequence: S'S - 1 times 2 times 3 times 4 times 5 times 6 times 7 times 8 times 9 times 10 times
132. Sequence: S'S' - 2 times 4 times 6 times 8 times 10 times
==========
1. Sequence: LLL - 4 times
2. Sequence: LLL' - 4 times
3. Sequence: LLR - 4 times

4. Sequence: LLR' - 4 times
5. Sequence: LLM - 4 times
6. Sequence: LLM' - 4 times
7. Sequence: LL'L - 4 times
8. Sequence: LL'L' - 4 times
9. Sequence: LL'R - 4 times
10. Sequence: LL'R' - 4 times
11. Sequence: LL'U - 4 times
12. Sequence: LL'U' - 4 times
13. Sequence: LL'D - 4 times
14. Sequence: LL'D' - 4 times
15. Sequence: LL'B - 4 times
16. Sequence: LL'B' - 4 times
17. Sequence: LL'F - 4 times
18. Sequence: LL'F' - 4 times
19. Sequence: LL'E - 4 times
20. Sequence: LL'E' - 4 times
21. Sequence: LL'M - 4 times
22. Sequence: LL'M' - 4 times
23. Sequence: LL'S - 4 times
24. Sequence: LL'S' - 4 times
25. Sequence: LRL - 4 times
26. Sequence: LRL' - 4 times
27. Sequence: LRR - 4 times
28. Sequence: LRR' - 4 times
29. Sequence: LRM - 2 times 4 times 6 times
30. Sequence: LRM' - 2 times 4 times 6 times
31. Sequence: LR'L - 4 times
…. and so on for 132 GB’s worth of text files.

Appendix D

TotalCubeStates

```
/* Name: Ryan McGregor
 * Thesis Project
 * Program Name: TotalCubeStates
 * Class: TotalCubeStates
 * Purpose of this Program:
 * This program exists to calculate the total number of states a Rubik's Cube of NxNxN
        *  size can exist in. The algorithm used to calculate this is the following:
        *
        *  F(n) = ((8! * 3^7)/24^((n + 1) mod 2)) * ((12! * 2^10)^(n mod 2)) *
        *                  (24!^⌊(n - 2)/2⌋) * ((24!/4!^6)^⌊((n - 2)/2)^2⌋)
        *
        * Where (8! * 3^7)/(24^(n + 1) mod 2) defines the total amount of possible
        * states for the corners of a Rubik's Cube to exist in,
        * ((12! * 2^10)^(n mod 2)) defines the total amount of possible states
        * for the middle edges of a Rubik's Cube to exist in,
        * (24!^⌊(n - 2)/2⌋) defines the total amount of possible states for
        * the wing edges of a Rubik's Cube to exist in, and
        * ((24!/4!^6)^⌊((n - 2)/2)^2⌋) defines the total amount of possible states
        * for X-Center Pieces, + Center Pieces, and Oblique Center Pieces to exist in.
        */
package cube;

import java.math.BigInteger;
import java.util.Scanner;

public class TotalCubeStates {

        static Scanner scan = new Scanner(System.in);

        //  Corners, defined as:
        //      (8! * 3^7)/(24^((n + 1) mod 2))
        public static BigInteger part1(int n){
                if(n == 0){
                        return new BigInteger("0");
                }
                if(n == 1){
                        return new BigInteger("1");
                }
                if(n % 2 == 0){//if n is even
                        return new BigInteger("3674160");// 7! * 3^6, or (8! * 3^7)/24
                }
                else
                        return new BigInteger("88179840");// 8! * 3^7
        }
```

```
//  Middle Edges, defined as:
//        (12! * 2^10)^(n mod 2)
public static BigInteger part2(int n){
        if(n == 1){
                return new BigInteger("1");
        }
        if(n % 2 != 0){//if n is odd
                return new BigInteger("490497638400");// 12! * 2^10
        }
        return new BigInteger("1");
}

//  Wing Edges, defined as:
//        24!^⌊(n - 2)/2⌋
public static BigInteger part3(int n){
        BigInteger answer = new BigInteger("1");
        if(n == 1){
                return answer;
        }
        int exponent = (n-2)/2;
        BigInteger base = new BigInteger("620448401733239439360000");          // 24!
        for(int i = 0; i < exponent; i++){
                answer = answer.multiply(base);
        }
        return answer;
}

//  X-Center Pieces, + Center Pieces, and Oblique Center Pieces,
//  defined as:
//        (24!/4!^6)^⌊((n - 2)/2)^2⌋
public static BigInteger part4(int n){
        BigInteger base = new BigInteger("3246670537110000");// 24!/4!^6
        BigInteger answer = new BigInteger("1");
        int exponent = ((n - 2) * (n - 2))/4;
        for(int i = 0; i < exponent; i++){
                answer = answer.multiply(base);
        }
        return answer;
}

public static BigInteger solver(int n){
        return part1(n).multiply(part2(n)).multiply(part3(n)).multiply(part4(n));
}
```

```
//  Defined as:
//  (4^6/2)^(n mod 2)
public static BigInteger SuperCubePart1(int n){
	BigInteger base = new BigInteger("2048");// 4^6/2
	int exponent = n % 2;
	BigInteger answer = new BigInteger("1");
	if(n == 1){
		return answer;
	}
	for(int i = 0; i < exponent; i++){
		answer = answer.multiply(base);
	}
	return answer;
}
//  Defined as:
//  (4!^6/2)^⌊((n - 2)/2)^2⌋
public static BigInteger SuperCubePart2(int n){
	BigInteger base = new BigInteger("95551488");// 4!^6/2
	BigInteger answer = new BigInteger("1");
	int exponent = ((n - 2) * (n - 2))/4;
	for(int i = 0; i < exponent; i++){
		answer = answer.multiply(base);
	}
	return answer;
}

public static BigInteger SuperSolver(int n){
	return solver(n).multiply(SuperCubePart1(n)).multiply(SuperCubePart2(n));
}
public static void main(String[] args){
	int i = 1;
	System.out.println("-------------------------------------------------------");
	while(i == 1){
		System.out.println("Enter a cube size: ");
		int n = scan.nextInt();
		while(n < 0){
			System.out.println("Cube size must not be negative");
			System.out.println("Enter a cube size: ");
			n = scan.nextInt();
		}
		System.out.println("Total " + n + "x" + n + "x" + n + " Cube Permutations:
" + solver(n));
```

```
                        //System.out.println("Total " + n + "x" + n + "x" + n + " SuperCube
Permutations: " + SuperSolver(n));
                        System.out.println("-------------------------------------------------------");
                        System.out.println("Would you like to see how many cube ");
                        System.out.println("permutations exist for another n-sized cube?");
                        String answer = scan.next();
                        if(answer.equalsIgnoreCase("yes") || answer.equalsIgnoreCase("y")){
                                i = 1;
                        }
                        else{
                                i = 0;
                        }
                        System.out.println("-------------------------------------------------------");
                }
        }
}
```

Sample Output

```
-----------------------------------------------------------
Enter a cube size:

User Input: 2

Total 2x2x2 Cube Permutations:     3674160
-----------------------------------------------------------
Would you like to see how many cube
permutations exist for another n-sized cube?

User Input: y

-----------------------------------------------------------
Enter a cube size:

User Input: 3

Total 3x3x3 Cube Permutations:     43252003274489856000
-----------------------------------------------------------
Would you like to see how many cube
permutations exist for another n-sized cube?

User Input: y

-----------------------------------------------------------
Enter a cube size:

User Input: 4

Total 4x4x4 Cube Permutations:     7401196841564901869874093974498574336000000000
-----------------------------------------------------------
Would you like to see how many cube
permutations exist for another n-sized cube?

User Input: n

-----------------------------------------------------------
```